



Deriving interactive Visual Association Maps From Documents Using Association Rule Mining

Bachelor's Thesis
of

Evangelos Michelioudakis
CSD 4950

as part of the study program

Curriculum: 2025-2026
at the

University of Crete
Computer Science Department

Advisor: Prof. Dr. Yannis Tztzikas 
Assistance: Prof. Dr. Dimitris Plexousakis 

Computer Science Department, DD.MM.20YY

Acknowledgments

First and foremost, I would like to express my sincere gratitude to my supervisor Prof. Dr. Yannis Tztzikas, for their continuous support, invaluable guidance, and immense patience throughout my research and the writing of this thesis. Their expertise and encouragement have been instrumental in the completion of this work.

I am also deeply grateful to my second supervisor, Prof. Dr. Dimitris Plexousakis for the time they dedicated to reviewing my work.

Furthermore, I would like to acknowledge the Department Of Computer Science at University Of Crete for providing an inspiring academic environment and the necessary resources to conduct this research.

Ευχαριστίες

Σε ένα πιο προσωπικό επίπεδο, το πιο μεγάλο και ουσιαστικό «Ευχαριστώ» ανήκει στα αγαπημένα μου πρόσωπα. Ιδιαίτερα, οφείλω ένα μεγάλο ευχαριστώ στην οικογένειά μου για την πολύτιμη συμπαράστασή της σε όλη τη διάρκεια των σπουδών μου. Τέλος, ένα 'ευχαριστώ' χρωστάω και στους φίλους μου, που στάθηκαν δίπλα μου σε όλη αυτή την προσπάθεια και έκαναν πιο υποφερτές τις ατελείωτες ώρες μπροστά στην οθόνη. Σας ευχαριστώ όλους από καρδιάς...

Contents

1	Introduction	6
1.1	General Discription	6
1.2	Preliminaries	7
1.2.1	Knowledge Graphs	7
1.2.2	Association Rule Mining (ARM)	7
1.2.3	System Metrics	8
1.2.4	Global Ranking (Weighted PageRank)	8
1.3	System's Purpose	8
2	System Functions	10
2.1	Basic System Functions / Processes	10
2.1.1	Adaptive Granularity and Text Segmentation	10
2.1.2	NLP Pre-processing and Indexing	11
2.1.3	Frequent Itemset Mining and Rule Generation	11
2.1.4	Taxonomy Construction and Hierarchy Leveling	11
2.1.5	Global Importance Ranking (PageRank)	12
2.1.6	Visualization and Explainability	12
3	Old Backend & New Backend	12
3.1	Backend On Both Sides	12
3.2	Algorithm Approach Change	15
3.2.1	A. The Legacy Approach: Apriori (Candidate Generation)	15
3.2.2	B. The Current Approach Eclat (Set Intersection)	15
3.3	Granularity Approach	16
3.3.1	A. The Legacy Approach: Static File-Level Granularity	16
3.3.2	B. The Current Approach: Dynamic Granularity Enhancement	16
3.4	Importance Approach	18
3.4.1	A. Version 1 (Legacy): Local Degree Centrality	18
3.4.2	B. Version 2 (Current): Global Weighted PageRank	19
4	Old Front-end & New Front-end	21
4.1	Front-end On Both Sides	21
4.2	Layout and Document Object Model (DOM) Structure	21
4.2.1	A. The Legacy Approach: Monolithic Linear Layout	21
4.2.2	B. The Current Approach: Split-Pane Dashboard (Component-Based)	22
4.3	The Physics-Based Rendering Engine	23
4.3.1	A. The Legacy Approach: Static Coordinates	24
4.3.2	B. The Current Approach: Force-Directed Graph Layout	24
4.4	Interactive State Management (The Control Loop)	25
4.4.1	A. The Legacy Approach: "Fire and Forget"	25
4.4.2	B. The Current Approach: Event-Driven Architecture	25
4.5	The Explainability Layer (Evidence Modal)	27
4.5.1	A. The Legacy Approach: Abstract Only	27

4.5.2	B. The Current Approach: Contextual Modal Windows	27
4.6	Advanced Visualization Modules	28
5	Data Description	30
5.1	Corpus Used For Testing	30
6	Design and Modeling	32
6.1	Domain Model And Data Structures	32
6.1.1	The Graph Data Transfer Object	32
6.1.2	The Semantic Node Entity	33
6.1.3	The Association Rule Abstraction	34
6.2	Application State Modeling	34
6.2.1	The Static Singleton State Pattern	34
6.3	RESTful Interaction Modeling	34
6.3.1	The Input Contract	35
6.4	User Interface Design Patterns	35
6.4.1	The Split-Pane Control Pattern	35
6.4.2	The Asynchronous Event-Driven Loop	35
6.5	Diagrams	36
7	System Architecture and Tools	38
7.1	Architecture and Design of the REST API	38
7.1.1	Files and Structure	38
7.1.2	Connection and Data Flow	38
7.1.3	Techologies And Tools	39
7.2	Design and Modeling of the Web Application	40
7.2.1	Files and Structure	40
8	UI Interface Update	41
8.1	New UI Style: Frontpage and New Dashboard Bar	41
8.2	Graph Style	43
8.3	Graph Details On-The-Spot Analysis	44
8.4	Live Filtering	45
9	System Testing	48
9.1	Web Application Testing	48
9.1.1	Preliminaries	48
9.2	System Experiments on macOS	49
9.2.1	SMCollection	49
9.2.2	20 News Groups Collection	51
9.2.3	MIMIC-III	52
9.2.4	Gutenberg Books	53
9.2.5	PDF Corpus	54
9.2.6	Full Experiment Results	55
9.3	System Experiments on Windows	56
9.3.1	SMCollection	56

9.3.2	20 News Groups Collection	57
9.3.3	MIMIC-III	59
9.3.4	Gutenberg Books	60
9.3.5	PDF Corpus	61
9.3.6	Full Experiment Results	63
10	System Usage Instructions	64
11	Conclusion	67
11.1	Summary	67
11.2	Deep Dive: Experimental and Empirical Comparisons	68
11.2.1	Performance Under Hardware Constraints (macOS M1)	68
11.2.2	Baseline Resource Efficiency (Windows i9)	69
12	Future Work	71
12.1	Semantic Expansion via Contextual Embeddings	71
12.2	Distributed Mining Architectures for Big Data	71
12.3	Advanced NLP and Temporal Graph Analysis	71
12.4	Human-in-the-Loop (Active Learning) Integration	72
12.5	Graph Interoperability and Exportation	72
12.6	LLM-Driven Cluster Summarization via Retrieval-Augmented Generation (RAG)	72
13	Bibliography	73

1 Introduction

1.1 General Discription

The primary objective of this Thesis is to substantially optimize and expand upon the previous work of E. Mitkousis regarding *Interactive Association Map Creation from Documents using Association Rule Mining*. [20] The central goal was to re-engineer the existing application, "*The Word Taxonomy Explorer*", transforming it into a high-performance, scalable text mining engine.

This transformation was achieved through a complete architectural overhaul of the backend. A critical improvement involves the migration from the computationally expensive *Apriori algorithm* to the highly efficient *Eclat (Equivalence Class Transformation) algorithm*. Unlike the legacy approach, which relied on horizontal data layouts and exhaustive candidate generation, the new system utilizes vertical transaction ID intersections to discover frequent itemsets, resulting in drastically reduced execution times. Furthermore, the system introduces a novel *Granularity Enhancement* module, allowing for dynamic text segmentation (sentence-level, paragraph-level, and semantic chunking) to improve the contextual accuracy of the mined rules.

Beyond the backend logic, the system's frontend has been modernized to enhance user experience. The outdated interface was refined with a responsive *HTML/CSS architecture*, featuring a new coloring profile for better visual hierarchy. Crucially, the interactivity has been upgraded to

support **Live Filtering** and **Evidence Retrieval**, allowing users to verify relationships by viewing the exact text snippets where concepts co-occur.

The developed system is a comprehensive **Text Mining Framework** designed to extract hierarchical taxonomies from unstructured text corpora. It employs a multi-stage pipeline that integrates Natural Language Processing (NLP), adaptive text segmentation, Association Rule Mining (ARM), and graph-based ranking algorithms. The architecture is designed to transform raw textual data into a directed, weighted graph that represents **Semantic Relationships Between Terms**.

The system is designed to ingest raw, unorganized text collections and transform them into a structured, hierarchical graph (a taxonomy) that reveals the semantic relationships between concepts.

1.2 Preliminaries

To understand the mechanics of the proposed system, it is necessary to define the fundamental concepts of **Association Rule Mining** and **Graph Theory** utilized in this study.

1.2.1 Knowledge Graphs

A **Knowledge Graph (KG)** is a structured representation of information, modeling entities and the relationships between them. In the context of this system, the taxonomy is constructed as a directed graph $G = (V, E)$, where:

- **Nodes (V):** Represent unique concepts or terms extracted from the text corpus (e.g., "Algorithm", "Java", "Optimization"). These are identified after NLP pre-processing steps such as *Stopword* removal and **Part-of-Speech tagging**.
- **Edges (E):** Represent the semantic rules or relationships between terms. An edge $A \rightarrow B$ implies a directional association, suggesting that the presence of term A **semantically** implies or introduces term B .

1.2.2 Association Rule Mining (ARM)

Association Rule Mining is a data mining technique used to discover frequent patterns, correlations, or causal structures among sets of items in transaction databases. In this system, a "transaction" is dynamically defined as a sentence, paragraph, or semantic chunk of text.

Formally, let $I = i_1, i_2, i_3, \dots, i_d$ be a set of literals (items). A transaction T is a set of items such that $T \subseteq I$. An association rule is an implication of the form $X \Rightarrow Y$, where $X \subset I$, $Y \subset I$ and $X \cap Y = \emptyset$

To evaluate the strength and validity of these rules, the system calculates three critical metrics: **Support**, **Confidence**, and **Lift**.

1.2.3 System Metrics

- **Support** Measures how frequently a term or itemset appears in the dataset. It indicates the popularity of the rule.

$$Support(X) = \frac{Frequency(X)}{|D|} \Rightarrow Support(X \Rightarrow Y) = P(X \cup Y) \quad [2] \quad (1)$$

Where D is the total number of transactions (documents, sentences, or chunks). A **high support** value indicates that the term is a common concept within the corpus. In our implementation, rules must meet a minimum user-defined support threshold to be considered for the taxonomy.

- **Confidence** Measures the reliability of the inference. It represents the conditional probability that the consequent Y appears in a transaction, given that the antecedent X is present.

$$Confidence(X \Rightarrow Y) = P(Y | X) = \frac{Support(X \cup Y)}{Support(X)} \quad [31] \quad (2)$$

A confidence of 0.8 means that in 80% of the cases where term X appears, term Y also appears. This metric is used as the primary weight for the edges in our graph, determining the strength of the relationship between parent and child nodes.

- **Lift:** In this system, Lift is used to **filter out coincidental rules** that occur simply because both terms are very common individually.

$$Lift(X \Rightarrow Y) = \frac{P(X \cup Y)}{P(X)P(Y)} \quad [1] \quad (3)$$

- If $Lift = 1$: X and Y are **independent** (no relationship).
- If $Lift > 1$: X and Y are **positively** correlated (the presence of X increases the probability of Y).
- If $Lift < 1$ X and Y are **negatively** correlated.

1.2.4 Global Ranking (Weighted PageRank)

While ARM identifies relationships, it does not inherently rank the importance of terms. To address this, the system applies a **Weighted PageRank** algorithm [24]. Unlike traditional node-degree heuristics, this algorithm iteratively calculates the importance of a node based on the importance of the nodes linking to it. The rank is distributed across edges proportional to the **Confidence** of the association, ensuring that semantically stronger links transmit more authority.

1.3 System's Purpose

The App has a **5-Way-Purpose**:

1. Unsupervised Knowledge Extraction

The system discovers relationships between terms without requiring pre-labeled training data. By using Association Rule Mining (Eclat algorithm), it identifies concepts that frequently co-occur, moving beyond simple keyword search to uncover hidden dependencies (e.g., if “Concept A” appears, “Concept B” is likely to appear).

2. Dynamic Context Definition

A core challenge in text mining is defining what constitutes a “transaction.” The system solves this through *Granularity Enhancement*, allowing users to dynamically choose the scope of context—entire document, single sentence, or “semantic chunk” (lexically similar sentence groups)—making the approach adaptable to diverse document types and analysis needs.

3. Automated Hierarchical Structuring

The system does not merely link terms; it organizes them vertically (general → specific).

- **Goal:** Distinguish broad topics (parents) from specific sub-topics (children).
- **Mechanism:** A heuristic based on graph topology (in-degree vs. out-degree) assigns a “Level” to each node, transforming flat association rules into a directed taxonomic graph.

4. Importance Ranking

Concepts are quantified by importance using a *weighted PageRank algorithm*. This produces a numerical rank for every node, enabling visualization to immediately highlight central, pivotal concepts versus peripheral ones.

5. Explainability and Verification

The system is fully transparent: an “Evidence Retrieval” endpoint allows users to click any relationship and instantly view the exact original text snippets that generated it, supporting rigorous validation of machine-generated insights.

In Summary The system is a tool for *Exploratory Data Analysis (EDA)* on text, designed to convert a “bag of words” into a “navigable map of concepts.”

2 System Functions

2.1 Basic System Functions / Processes

The *System Architecture and Methodology* is composed by **Five (5)** distinct functional modules:

2.1.1 Adaptive Granularity and Text Segmentation

The initial stage of the pipeline focuses on transforming raw documents into a *transactional format suitable for mining*. Unlike traditional approaches that rely on fixed delimiters, this system implements a `GranularityEnhancement` module that allows for *dynamic definition of a "transaction."* The system supports four levels of **Granularity**:

- **DOCUMENT Level:** Treating the entire file as a single transaction.
- **PARAGRAPH Level:** Splitting content based on double line breaks.
- **SENTENCE Level:** `BreakIterator` for locale-aware sentence boundary detection.
- **SEMATIC CHUNK:** An advanced method that groups sentences into "chunks" **based on lexical similarity**. This uses the *Jaccard similarity* coefficient to measure the overlap of terms between adjacent sentences; if the similarity drops below a **user-defined threshold**, a new chunk is initiated.

Table 1: Comparison of Transaction Definition: Sentence vs. Semantic Chunking

Level	Transaction Boundaries	Reasoning & Logic
Original Text	<i>"Machine learning optimizes taxonomies. These algorithms use statistical patterns. In contrast, history studies human events".</i>	The raw input contains three distinct sentences with a thematic shift between the second and third
SENTENCE	T1: <i>"Machine learning optimizes taxonomies."</i> T2: <i>"These algorithms use statistical patterns."</i> T3: <i>"In contrast, history studies human events."</i>	Uses <code>BreakIterator</code> with <code>Locale.US</code> to identify periods. Each sentence is an independent transaction; co-occurrence is only tracked within these limits.
SEMATIC CHUNK	T1: <i>"Machine learning optimizes taxonomies. These algorithms use statistical patterns."</i> T2: <i>"In contrast, history studies human events."</i>	The system calculates $J(S_1, S_2) \geq \theta$ because <i>"algorithms"</i> relates to <i>"Machine learning"</i> . However, $J(S_1, S_2) \leq \theta$ triggers a <i>"Semantic Shift"</i> because there is no lexical overlap, closing the transaction.

2.1.2 NLP Pre-processing and Indexing

Once segmented, the data undergoes rigorous **pre-processing** to filter noise and retain semantically significant terms. The `Index` and `POSCheck` modules handle this phase:

1. Tokenization And Stopword Removal:

Input text is sanitized to remove non-alphanumeric characters and filtered against a pre-defined list of stopwords.

2. Part-of-Speech (POS) Filtering:

The system integrates the *Stanford CoreNLP pipeline* to tag tokens. It specifically filters for nouns (`NN`), verbs (`VB`), and adjectives (`JJ`), while explicitly excluding numerical values to ensure the taxonomy is built on conceptual terms rather than data points.

3. TF-IDF Scoring:

To identify the most representative terms in the corpus, the system calculates *Term Frequency-Inverse Document Frequency (TF-IDF)*. Only the top- K terms (sorted by TF-IDF score) are retained for the mining phase to **reduce** computational complexity and **improve** relevance.

2.1.3 Frequent Itemset Mining and Rule Generation

The core relation extraction is performed using Association Rule Mining. The system implements the *Eclat algorithm (Equivalence Class Transformation)*, a depth-first search algorithm that is generally more memory-efficient than *Apriori* for this context.

Recursive Mining: The system identifies frequent itemsets by recursively intersecting Transaction ID (**TID**) sets. It supports finding itemsets up to a specified maximum phrase length (**depth**).

Rule Derivation: From the frequent itemsets, the system derives *Association Rules* ($A \rightarrow B$). These rules are filtered based on a **Minimum Confidence Threshold**. Additionally, the system calculates *Lift*, a metric that measures how much more likely B is to co-occur with A than would be expected by chance, providing a measure of the rule's strength.

2.1.4 Taxonomy Construction and Hierarchy Leveling

The generated rules are transformed into a *Graph Structure* where *terms become nodes* and *rules become directed edges*. A crucial component of this system is the automatic determination of

hierarchy levels to structure the graph as a **taxonomy**:

- **Graph Construction:** Nodes are instantiated for *antecedents* and *consequents*. **Directed edges** are created based on the mined rules, weighted by *Confidence* and *Lift*.
- **Dynamic Level Calculation:** To approximate a hierarchical structure (Parent vs. Child), the system calculates a `level` attribute for each node. This is derived using a heuristic formula based on the node's connectivity:

$$Level \approx \alpha \times InDegree - \beta \times OutDegree \quad (4)$$

This logic places broad, general terms (**high out-degree, low in-degree**) at the top of the hierarchy and specific terms (**high in-degree**) at the bottom.

2.1.5 Global Importance Ranking (PageRank)

To further distinguish the importance of terms within the generated taxonomy, the system applies a **Weighted PageRank Algorithm**.

- **Iterative Ranking:** The algorithm iteratively updates the **rank** of each node based on the ranks of its incoming links.
- **Weighted Distribution:** Unlike standard PageRank, this implementation utilizes the *confidence of the association rules as weights*. A node distributes its rank to its children proportional to the *confidence* of the link connecting them, **ensuring that stronger semantic relationships have a greater influence on the final ranking**.

2.1.6 Visualization and Explainability

The final output is exposed via a **REST API** as a `GraphDt` object, designed for frontend visualization. A unique feature of this system is its *"Evidence Retrieval"* capability. For any given edge in the **taxonomy**, the user can query the system to retrieve specific text snippets (evidence) from the original corpus that contain both the *antecedent* and *consequent* terms. This **ensures the taxonomy is transparent and verifiable**.

3 Old Backend & New Backend

3.1 Backend On Both Sides

The most significant architectural upgrade in the backend is the **transition from a Horizontal, Breadth-First mining strategy (Apriori) to a Vertical, Depth-First strategy (Eclat)**. This shift addresses the computational bottlenecks inherent in *mining high-dimensional text data*. One more equally important upgrade can be observed in the evolution of the **Transaction Definition** within the system. In **Association Rule Mining**, the definition of a "transaction" - *the specific window*

of text considered for analysis - determines the quality of the discovered rules. The system has evolved from a coarse, static definition to a dynamic, semantically aware **segmentation engine**. The transition from Version 1 (Legacy) to Version 2 (Current) represents a shift from "File-Based Processing" to "Content-Aware Segmentation". This change was implemented to eliminate false associations caused by arbitrary text adjacency. The third change can be located in the mathematical and structural evolution of the **Taxonomy Ranking** module. The system has moved from a locally defined heuristic, which organized terms based solely on their immediate connectivity, to a **globally optimized ranking** algorithm that measures the semantic authority of each concept within the entire graph. The goal of the **ranking module** is to assign a numerical importance score to every term (node) in the taxonomy. This score determines the hierarchy: "Parent" nodes (general concepts) are distinguished from "Child" nodes (specific sub-concepts).

Before diving deeper in the **Backend** changes, addition that where implemented in the Legacy App, some observation must be done in the core logic of the bigger change **Apriori** → **Eclat**. In more detail, how do they both work:

- **Apriori:** The initial version of the system utilized the **Apriori Algorithm**, a seminal method in *Association Rule Mining* proposed by Agrawal and Srikant. It employs a **Breadth-First Search (BFS)** strategy and a horizontal database layout to discover frequent itemsets. The core principle of the algorithm is the Apriori Property (Downward Closure Property), which states:

"All non-empty subsets of a frequent itemset must also be frequent."

Conversely, if an itemset is infrequent, all its supersets are also infrequent and can be pruned from the search space.

Table 2: Horizontal Data Layout Of Apriori

TID	Items
T_1	{ Association, Rule, Mining }
T_2	{ Association, Map }
T_3	{ Rule, Mining, Algorithm }
T_4	{ Taxonomy, Extraction }

Table 2 Description: **TID** == Transaction and **Items** == Terms extracted from text. This horizontal format requires the system to **scan all items within each transaction** to verify the presence of a specific candidate, leading to the "Candidate Explosion" problem during high-depth mining [1].

Operational Flow: The algorithm executes in iterations ($k = 1, 2, \dots$):

1. **Candidate Generation:** The system generates candidate itemsets of length k by joining frequent itemsets of length $k - 1$. This was implemented in the legacy system using combinatorial logic.

2. **Support Counting:** For every candidate set generated, the system must scan the entire transaction database to count its occurrence.

3. **Pruning:** Candidates that fail to meet the minimum support threshold are discarded.

Limitations in the Legacy System: While conceptually simple, **Apriori** suffers from a massive I/O bottleneck. As the length of the phrases (k) increases, the number of candidate sets can grow exponentially. In the previous implementation, this led to extreme computational overhead, often resulting in the system failing to mine deep relationships due to the "*Candidate Explosion*" problem.

- **Eclat:** To address the performance limitations of **Apriori**, the current system implements the **Eclat** (Equivalence Class Transformation) Algorithm. *This represents a paradigm shift from horizontal to vertical data mining.*

Table 3: Vertical Data Layout Of Eclat

Term (i)	TID_set ($t(i)$)
Data	{ 1, 2 }
Mining	{ 1, 3 }
Eclat	{ 1, 3 }

Table 3 Description: The vertical layout enables the Eclat algorithm to perform **recursive mining through Transaction ID intersections in memory**, avoiding the I/O bottlenecks associated with traditional document scanning [33].

Operational Flow: Eclat employs a **Depth-First Search (DFS)** strategy and eliminates the need for candidate generation entirely.

1. **Intersection-Based Counting:** To calculate the support of a new itemset A, B , the algorithm simply computes the intersection of their TID lists:

$$TID(A \cup B) = TID(A) \cap TID(B) \quad (5)$$

The size of the resulting intersection set is the support of $\{A, B\}$.

2. **Recursive Mining:** The algorithm recursively intersects TID lists to discover longer patterns. If the intersection of two lists is empty (or below the threshold), the branch is immediately pruned.

Advantages in the New Architecture:

- **No Database Rescanning:** The support is calculated via efficient set arithmetic in memory, without repeatedly scanning the raw text files.
- **Memory Efficiency:** By processing data vertically, the system avoids generating the millions of invalid candidates that plagued the legacy Apriori implementation.
- **Speed:** The Depth-First approach allows the system to quickly find long, specific phrases (e.g., "*Association Rule Mining Algorithm*") without waiting to enumerate all shorter phrases first.

3.2 Algorithm Approach Change

3.2.1 A. The Legacy Approach: **Apriori** (Candidate Generation)

- **Data Layout (Horizontal):** The legacy system processed data in a *horizontal format*, where the database is a list of transactions, and **each transaction contains a set of items (words)**.
- **Algorithmic Logic:** The old `Index.java` implemented an iterative "*Generate-and-Test*" methodology:

1. Candidate Generation:

To find frequent phrases of length k , the system first generated all **possible combinations** of frequent $(k - 1)$ -itemsets.

2. Support Counting:

For *every* candidate generated, the system had to scan *the entire collection of documents again* to verify if the candidate actually appeared enough times.

- **The Bottleneck:** The *complexity* was exponential $O(2^d)$, where d is the **maximum depth** of the taxonomy. As seen in the legacy code, the `Findsubsets` method and `getCombinations` function created a *massive overhead*. If the text contained a frequent phrase of 10 words, **Apriori** would attempt to generate and verify **ALL** $2^{10} - 1$ subsets, leading to the "**TOO MANY SUBSETS**" error handling seen in the old code.

3.2.2 B. The Current Approach **Eclat** (Set Intersection)

- **Data Layout (Vertical):** The new system utilizes the *Eclat (Equivalence Class Transformation)* algorithm. Instead of looking at files as lists of words, *it inverts the data into TID Lists (Transaction ID Lists)*.
 - *Concept: "Word A" $\rightarrow$ List of Sentence IDs where A appears.*
- **Algorithmic Logic:**

1. No Candidate Generation:

Eclat does not "**guess**" candidates. *It discovers them.*

2. Intersection:

To determine the *support* of the pair (A,B) , the algorithm simply **computes the intersection of their TID lists**:

the closure operator satisfies the **monotonicity property**

As is standard for closure operators in formal concept analysis, our operator T is *monotonic*: $T(A \cup B) = T(A) \cap T(B)$ for all concept sets A and B .

3. Depth-First Search (DFS):

The algorithm *recursively* intersects lists to find longer patterns **immediately, rather than waiting to complete all patterns of the current length**.

- **Performance Gain:**
 - **Memory Efficiency:** Since text data is sparse (*most words do not appear in most sentences*), **the vertical intersection is extremely fast compared to scanning full text files**.
 - **Scalability:** *Eclat* skips non-existent patterns entirely. If $T(A) \cap T(B)$ is empty, **the algorithm immediately stops exploring that branch**, avoiding the wasted computation seen in the legacy `Findsubsets` loop.

3.3 Granularity Approach

3.3.1 A. The Legacy Approach: **Static File-Level** Granularity

- **Implementation:** The text processing logic was embedded directly inside the `VisualRestController`. It utilized a crude whitespace splitting mechanism: `String[] tokens = content.split("\\s+");`.
- **The "Bag-of-Words" Flaw:** This approach treated the entire document as a single transaction.
 - **The Problem:** It ignored semantic boundaries. The last word of Chapter 1 was treated as strictly *"adjacent"* to the first word of Chapter 2.
 - **Result:** This introduced significant noise, creating *"False Positive"* associations between terms that appeared in the same file but had no conceptual relationship (e.g., a term in the Header linking to a term in the Footer).

3.3.2 B. The Current Approach: **Dynamic** Granularity Enhancement

The current system introduces a dedicated singleton-style class, `GranularityEnhancement`, which decouples segmentation logic from mining logic. It allows the user to define the *"Context Window"* based on the analytical goal, offering **four distinct levels of granularity**:

1. Sentence-Level Granularity (High Precision)

- **Logic:** Instead of simple regex splitting, the system employs Java's `BreakIterator` class initialized with `Locale.US`.
- **Advantage:** This ensures linguistic accuracy. It correctly handles complex boundaries (*e.g., distinguishing the period in "Dr. Smith" from the end of a sentence*).
- **Effect:** Transactions are strictly limited to single sentences. A rule $A \rightarrow B$ is only mined if A and B occur **within the same sentence structure**.

2. Paragraph-Level Granularity (Thematic Context)

- **Logic:** The system utilizes a regex pattern `(\\r?\\n){2,}` to identify double line breaks, which typically **denote paragraph transitions**.
- **Effect:** This captures **medium-range dependencies**. It allows terms to be associated if they are discussed within the same argument block, even if they are in different sentences.

3. Document-Level Granularity (Global Context)

- **Logic:** Retains the legacy capability `(content.split("\\s+"))` for backward compatibility or when analyzing very short texts (*e.g., tweets or abstracts*) where the entire text represents a **single thought**.

4. Semantic Chunking (Mathematical Segmentation)

The most advanced feature of the backend update is *Semantic Chunking*. This method dynamically sizes the transaction window based on **topic continuity** rather than fixed punctuation.

- **The Algorithm:** The `semanticChunking` method iterates through the text **sentence by sentence**. It maintains a "current chunk" and decides whether to extend it or close it based on the lexical similarity between the current sentence S_{curr} and the previous sentence S_{prev} .
- **Mathematical Model (Jaccard Similarity):** To quantify the relationship between two sentences, the system calculates the **Jaccard Similarity Coefficient (J)**. Let $W(S)$ be the set of unique words in a sentence S . The similarity is calculated as:

$$J(S_{prev}, S_{curr}) = \frac{|W(S_{prev}) \cap W(S_{curr})|}{|W(S_{prev}) \cup W(S_{curr})|} \quad (6)$$

- **The Threshold Logic:** The system defines a threshold θ (`chunkThr`).
 - If $J(S_{prev}, S_{curr}) \geq \theta$: The topic is continuous. S_{curr} is added to the **current transaction**.
 - If $J(S_{prev}, S_{curr}) < \theta$: A *semantic shift has occurred*. The current transaction is **closed**, and a new transaction begins with S_{curr} .
- **Impact:** This architectural change ensures that the taxonomy is built on **Semantic Co-occurrence** rather than Physical Proximity. By mathematically ensuring that transactions only contain lexically related sentences, the confidence scores of the resulting rules reflect genuine conceptual links.

Table 4: Logic Gate for Semantic Transaction Definition [13, 16]

Condition	System Action	Semantic Interpretation
$J \geq \theta$	$S_{curr} \in \text{Current Chunk}$	Topic continuity is maintained; sentences are lexically related [13].
$J < \theta$	New Chunk $\leftarrow S_{curr}$	A semantic shift is detected; a new transaction window begins [13].

3.4 Importance Approach

3.4.1 A. Version 1 (Legacy): Local Degree Centrality

In the legacy implementation, the importance of a node was calculated using a static formula based on **Degree Centrality**. This metric is "local" because it only considers the immediate neighbors of a node, ignoring the wider network structure.

- **The logic:** A node was considered "**General**" (high level) if it had many outgoing links (*it implies many things*) and few incoming links (*it is implied by few things*).
- **The Mathematical Model:** The system utilized a linear combination of In-Degree and Out-Degree, as implemented in the legacy `Node.java` file:

$$Level(n) \approx \alpha \times |InDegree(n)| - \beta \times |OutDegree(n)| \quad (7)$$

- Where α and β are user-defined constants.
- $InDegree(n)$: Number of terms implying n .
- $OutDegree(n)$: Number of terms implied by n .
- **The Limitations:**
 - **Susceptibility to Noise:** A frequent but irrelevant term (e.g., a common stopword that slipped through filtering) could have a massive Out-Degree, artificially inflating its rank.
 - **Lack of Context:** A node connected to 10 "garbage" nodes was ranked higher than a node connected to 5 "critical" concepts. The quality of the connections was ignored.

3.4.2 B. Version 2 (Current): Global Weighted PageRank

The current system implements a **Weighted PageRank algorithm**. This is a "global" metric because the rank of a node is determined recursively by the rank of the nodes pointing to it. A term is considered important not just because it has connections, but because *other important terms refer to it*.

- **The Logic:** The system models a "Random Surfer" traversing the taxonomy.
 - The surfer moves from term A to term B based on the **Confidence** of the rule $A \rightarrow B$.
 - If the association is strong (high confidence), the probability of traversing that link increases.
- **The Mathematical Model:** Based on `PageRank.java` implementation, the rank $PR(u)$ for a node u is calculated iteratively:

$$PR(u) = \frac{1-d}{N} + d \sum_{v \in In(u)} \left(PR(v) \times \frac{Confidence(v \rightarrow u)}{\sum_{k \in Out(v)} Confidence(v \rightarrow k)} \right) \quad (8)$$

- d : The **Damping Factor** (base value of 0.85), representing the **probability** that the surfer continues clicking links rather than jumping randomly.
- N : The total numbers of *Nodes* in the Taxonomy.
- $In(u)$: The set of parent nodes pointing to u .
- **The Weighting Factor:** $\frac{Confidence(v \rightarrow u)}{\sum_{k \in Out(v)} Confidence(v \rightarrow k)}$
Unlike standard **PageRank** (which splits rank evenly), your system distributes the parent's rank proportionally to the confidence of the association rules. A stronger rule transmits more "authority" to the child node.

- **Robustness Features (Dangling Nodes):** This implementation explicitly handles "Dangling Nodes" (terms with no children/outgoing links). In standard **PageRank**, these nodes absorb rank and "leak" it out of the system. The algorithm detects nodes where `totalWeight == 0` and redistributes their rank equally among all nodes in the graph ($\frac{\text{distributedRank}}{N}$), **ensuring the mathematical stability of the ranking**.

Table 5: Evolution of Term Importance Ranking

Feature	Local Centrality	Global Weighted PageRank
Scope	Local (immediate neighbors only) [8].	Global (recursive network authority) [23].
Weighting	Unweighted degree counts [8].	Weights proportional to Rule Confidence [32].
Noise Resistance	Susceptible to frequent irrelevant terms [22].	Filters "garbage" nodes via recursive importance [23, 32].
Stability	Fixed formula [8].	Handles "Dangling Nodes" via rank redistribution [23].

4 Old Front-end & New Front-end

4.1 Front-end On Both Sides

In the evolving landscape of software engineering, particularly within applications leveraging **Knowledge Graphs (KGs)** for structured data representation and semantic querying, the front end emerges as a pivotal component that *bridges* complex backend functionalities with end-user accessibility. This preliminary analysis posits that modifications to an application's front end—encompassing visual redesigns, navigational improvements, and interactive augmentations are not merely cosmetic but essential for optimizing user engagement and operational efficacy. Drawing from foundational principles in **human-computer interaction (HCI) and user experience (UX)** design, we explore why the front end must be clear and aesthetically appealing. Thus, a **visual** as well as an **interactive** update was essential in the previous approach.

- **The Main Goal:** The focus, was on providing the user with a more clear page, **easier** to navigate and provide the needed metrics for the App. Synonym to that, is also **Interactivity** meaning the ability for the user to **view**, **filter**, **edit** and **understand** the results in different more comprehensive ways. The evolution of the application's front-end marks a **transition** from a static "*Result Viewer*" to a "*Visual Analytics Dashboard*." This transformation was necessary to support the increased complexity of the backend mining algorithms (Eclat and PageRank). The new interface utilizes a **Single-Page Application (SPA)** behavior model to maintain state without requiring full page reloads.

Following those **preliminaries**, the visual changes follow a many steps of updates. From the "*simple*" update in the core HTML/CSS code of the page, to the more "*complex*" `JavaScript` sections of code added for the live filters and different views to work coherently and in a unite way:

4.2 Layout and Document Object Model (DOM) Structure

The structural overhaul of the front-end was driven by the need to support high-dimensionality data without overwhelming the user. While the backend logic in `VisualRestController` became more complex (handling Eclat mining and PageRank), the front-end needed to decouple the *visualization* from the *configuration*.

This led to a shift from a linear, document-based layout to a **Dashboard-Oriented Architecture**, utilizing a responsive DOM and a semantic CSS framework.

4.2.1 A. The Legacy Approach: Monolithic Linear Layout

- **DOM Structure:** The Layout consisted of a single vertical column. The configuration forms were stacked on the bottom of the visualization canvas.

-
- **User Experience (UX) Bottleneck:** his linear structure created a "Scroll Gap". To adjust a parameter, the user had to scroll down, lose sight of the graph, submit changes, and then scroll back up to interpret the results. The layout treated the application as a static report rather than an interactive tool.

4.2.2 B. The Current Approach: Split-Pane Dashboard (Component-Based)

The new version implements a **View-Controller Pattern** directly in the DOM. The `VisualController` serves a lightweight HTML shell (visualize endpoint), which serves as the container for dynamic components.

- **The Control Plane (Sidebar):** The layout introduces a fixed-position sidebar dedicated to the `ValueRequest` parameters (*Support, Confidence, Granularity*).
 1. **DOM Behavior:** This element is removed from the document flow, ensuring it remains accessible regardless of the graph's size or zoom level.
 2. **Benefit:** This enables **Simultaneous Tuning**. The user can tweak the "Confidence" slider while keeping their eyes fixed on the graph, observing changes in real-time.
- **The Visualization Plane (Main Viewport):** The main content area is a flexible container designed to render the `GraphDt JSON` object.
 1. **Dynamic Injection:** Unlike the legacy version, the graph nodes are not hard-coded in the HTML. The DOM is initially empty. Upon receiving the `GraphDt` response, `JavaScript` dynamically injects *SVG* or *Canvas* elements into this viewport.
 2. **Z-Index Layering:** The layout utilizes CSS Z-Indexing to manage overlay views. The **Heatmap** and **Chord Diagram** are structured as modal overlays that sit on top of the main graph, allowing users to toggle between views without navigating away from the core dataset.
- **CSS Architecture: Data-Driven Styling**

The CSS in the new version is not merely decorative; it is functionally bound to the data properties calculated by the backend algorithms.

Hierarchical Coloring Profile:

The most significant CSS improvement is the **Semantic Color Mapping**. The back-end `Node` class calculates a specific hierarchy level based on the graph topology. The `GraphDt` object passes this `level` (an integer) to the front-end.

- **Implementation:** The CSS defines a spectrum of classes corresponding to these levels.
 1. **High-Level Nodes (Roots):** Assigned distinct, saturated colors (*e.g.*, *Deep Red* or *Navy*) to signify broad categories.
 2. **Mid-Level Nodes:** Assigned transitional hues.
 3. **Low-Level Nodes (Leaves):** Assigned lighter, *de saturated* tones.
- **Result:** This allows the user to intuitively understand the "Depth" of a concept in the taxonomy simply by its color, without needing to read the label.

Responsive Scaling (Fluid Grid):

To accommodate the complex visualization on different devices, the CSS utilizes **Flexbox** and **CSS Grid**.

- The container for the graph allows for fluid resizing. If the browser window is resized, the force-directed simulation recalculates the center of gravity, and the CSS ensures the *SVG* elements scale proportionally, preventing nodes from being cut off or distorted.

Interactive Feedback States:

The CSS now includes pseudo-classes for interaction (`:hover`, `:active`) applied to the graph nodes.

- When a user hovers over a node, CSS transitions are triggered to dim unrelated nodes and highlight the connected neighbors. This "*Spotlight Effect*" reduces cognitive load by visually filtering noise, a feature absent in the static legacy version.

4.3 The Physics-Based Rendering Engine

Visualizing **hundreds, thousands** of association rules requires a sophisticated layout algorithm to prevent "*Visual Clutter*" (*the hairball problem*).

4.3.1 A. The Legacy Approach: Static Coordinates

- **Rendering:** `Nodes` were likely placed using simple coordinate calculations or basic randomization.
- **Problem:** Without a physics simulation, nodes often overlapped. High-degree nodes (parents) were visually indistinguishable from leaf nodes, making the taxonomy structure invisible to the naked eye.

4.3.2 B. The Current Approach: Force-Directed Graph Layout

- **Simulation:** The front-end now employs a **Force-Directed Algorithm** (*Barnes-Hut simulation*).

1. **Repulsion:** `Nodes` act like charged particles, repelling each other to *prevent overlap*.
2. **Spring Tension:** `Edges` act like springs. *Stronger* connections (higher Confidence from `Edge.java`) pull nodes closer together.
3. **The physics simulation** calculates the optimal placement of nodes by minimizing the system's total visual "energy." This is achieved by iteratively applying two primary physical forces to the graph:

- **Repulsive Forces (Coulomb's Law):** To prevent visual clutter, all nodes act as similarly charged particles that repel each other. The repulsive force F_r between any two nodes u and v separated by a Euclidean distance of $d(u, v)$ is calculated as:

$$F_r(u, v) = \frac{k^2}{d(u, v)} \quad (9)$$

where k represents the optimal spring length (the ideal baseline distance between nodes based on the canvas area).

- **Attractive Forces (Hooke's Law):** The directional edges between related terms act as springs, pulling connected concepts together. Crucially, in this system, the physical tension of the spring is mapped directly to the statistical strength of the rule. The attractive force F_a is defined as:

$$F_a(u, v) = w_{(u, v)} \cdot \frac{d(u, v)^2}{k} \quad (10)$$

where $w_{(u, v)}$ represents the weight of the edge (specifically, the *Confidence* metric of the association rule).

Through continuous iterations, these opposing forces reach an equilibrium, organically revealing distinct semantic clusters where tightly coupled concepts physically gravitate toward one another.

- **Result:** The graph "*settles*" into a natural state where clusters of related terms emerge organically.
- **Visual Encoding:**

-
1. **Size:** `Node` size is dynamic, bound to the `PageRank` score calculated in the backend. Important terms appear *physically larger*.
 2. **Color:** The level attribute from `Node.java` is mapped to a color gradient, creating a visual heatmap of the hierarchy (Root → Leaf).

4.4 Interactive State Management (The Control Loop)

4.4.1 A. The Legacy Approach: "Fire and Forget"

- **Data Flow:** User sets *Parameters* → *Submit Form* → *Server Processes* → *New Page Loads*.
- **Latency:** The "*Feedback Loop*" was slow. Users could not experiment with sensitive parameters like **Granularity** because the cost of reloading was too high.

4.4.2 B. The Current Approach: Event-Driven Architecture

The core innovation in the system's interactivity lies in its "*Event-Driven Control Loop*". Unlike the legacy system, which required a full application restart to change mining parameters, the current architecture implements a dynamic **State Injection** mechanism. This allows the user to filter the taxonomy in real-time by injecting new thresholds (*Support and Confidence*) directly into the running backend process.

- **The Communication Protocol** (`ValueRequest` *DTO*). The interaction begins when the user manipulates a control (e.g., *sliding the "Minimum Confidence" bar*) on the front-end. This action triggers the construction of a `JSON` payload that mirrors the backend's internal state requirements.
- **The Data Carrier:** The system utilizes a dedicated *Data Transfer Object (DTO)* class, `ValueRequest.java`, to formalize this communication. This object encapsulates the user's intent, transporting the critical mining parameters:
 1. `support:` The frequency threshold.
 2. `confidence:` The strength threshold for rules.
 3. `phrase _ length:` The maximum depth for itemset mining.
- **The Endpoint:** This payload is transmitted via a *RESTful* `POST` request to the `/api/submit-values` endpoint managed by the `VisualRestController`.
- **Backend State Storage (Static Persistence)** Upon receiving the request, the backend does not simply pass the values to a function; it updates the **Global System State**.
- **In-Memory Storage:** The `VisualRestController` maintains the application state using static class variables.
 1. `private static double support;`

-
2. `private static double confidence;`
 3. `private static HashMap<String, Rule> rules;`
- **Mechanism:** When `EnterValues` is called, the incoming parameters from the `ValueRequest` immediately overwrite these static variables. This persistence model ensures that the mining engine always operates on the most current user-defined configuration without needing to pass parameters down through a long chain of method calls.
 - **The Live Filtering Engine (The Process Loop)** The "Live Filtering" effect is achieved through the re-execution of the `newProcess()` method, which acts as the system's logic gate.
 1. **Confidence Slider / Support Slider:** The user can *live* change the `Confidence` or `Support` value, if the visualization is a "hairball". The changes, are applied without the need of the rerunning of the whole program.
 2. **Max Edges Per Node (K):** This filter is used to, furthermore, help the work of **PageRank**. Once the original graph has been created, the user can *change the value of the Top-K edges*. Resulting, in a more *purified and clear* representation of the most **interesting data** - according to `Confidence` -.
 - **Rule Regeneration:** Once the new parameters are stored, the system invokes `newProcess(I)`. Crucially, this method initializes a fresh rules `HashMap` (`rules = new HashMap<>()`), clearing all previous data.
 - **Threshold Logic:** The system iterates through the frequent itemsets discovered by the Eclat algorithm (`index.Findsubsets`). For every potential relationship, it performs a real-time check against the newly injected state:


```

if(conf >= confidence) {
    // Create Rule
}
      
```

This conditional block acts as the *filter*. If the calculated confidence of a potential rule falls even slightly below the user's new setting, it is immediately discarded and never reaches the graph generation stage.
 - **Graph Reconstruction** (`initFrame`) Finally, the visualization state is rebuilt from scratch. The `initFrame()` method iterates through the filtered rules map to construct the `GraphDt` object.
 - **Dynamic Topology:** Nodes and edges are instantiated only if they survived the filtering process.
 - **Hierarchy Recalculation:** As the topology changes (*due to stricter filtering removing edges*), the system dynamically recalculates the level of every node (`setLevels()`), ensuring that the visual hierarchy (colors and layout) accurately reflects the current subset of data, not the original full dataset.

4.5 The Explainability Layer (Evidence Modal)

A critical addition for **Research** rigor is the ability to drill down from abstract nodes to concrete text.

4.5.1 A. The Legacy Approach: Abstract Only

- **View:** The user saw a visual representation on the graph with the two words connected. When clicked on a **Node** a small pop-up box with the simple information: "*Word1* $\rightarrow$ *Word2*" on the screen.
- **Blind Spot:** The user had no way of knowing from which *document, sentence or section of text* generated this rule.

4.5.2 B. The Current Approach: Contextual Modal Windows

- **Explainability & Architecture** A major limitation in many text mining systems is the "*Black Box*" problem: the algorithm asserts a relationship exists (*e.g.*, $A \rightarrow B$), but the user cannot verify **Why**. The **Explainability Layer** addresses this by providing an "*Audit Trail*" for every edge in the taxonomy. This is implemented via a dedicated verification pipeline involving specific DTOs and backend retrieval logic. Unlike the graph generation, which pre-calculates the entire taxonomy, the Evidence Layer operates on an **On-Demand (Lazy Loading)** basis. We do not send all text snippets to the front-end initially, as this would cause performance degradation due to payload size. Instead, we utilize a targeted query mechanism.
 - **The Trigger:** When a user interacts with a specific edge in the visualization (*e.g.*, *clicking the line connecting "Machine" and "Learning"*), the front-end captures the unique identifiers of the **Antecedent** (U) and the **Consequent** (V).
 - **The Request:** An asynchronous call is dispatched to the backend, specifically requesting "*proof*" for the relationship $U \rightarrow V$.
 - **The Data Transfer Object** (`EvidenceResponse`):

```
public class EvidenceResponse {  
    private List<String> snippets;  
    // Constructor and Getters  
}
```

The `snippets` list contains the extracted raw text segments from the original documents. By encapsulating this in a rigid Java class, we ensure the front-end always receives a predictable JSON structure, preventing parsing errors.

- **The Retrieval Logic:** On the backend, the `VisualRestController` handles the retrieval process. Because the system retains access to the raw file paths and content via the `readFileAsString` method, it can revisit the source material.

Search Algorithm:

1. The system identifies which documents (or "*transactions*") contributed to the support count of the rule $U \rightarrow V$.
2. It scans these specific text segments to extract the context window (*usually the sentence or paragraph*) where both terms co-occur.
3. This verifies that the relationship is based on **Proximity** and **Context**, not just statistical frequency.

- **The User Experience: The Evidence Modal:** The front-end presents this data in a non-intrusive Modal Window (*an overlay dialog*), preserving the user's mental model of the graph.

- **Contextual Display:** The modal lists the retrieved snippets. For example:

"..algorithms such as **Support Vector Machines** are often used for *classification* tasks..."

- **Visual Highlighting:** The front-end dynamically parses these strings to bold or highlight the query terms (U and V). This allows the user to instantly validate the semantic sense of the rule (*e.g., confirming that "Apple" refers to the company, not the fruit, based on the surrounding words like "iPhone" or "Stock"*).

Table 6: Evolution of Front-end Architecture and Layout

Feature	Legacy: Monolithic Linear	Current: Split-Pane Dashboard
Structure	Single vertical column; forms at bottom [7].	Fixed sidebar control plane; main visualization viewport [7].
Interaction	"Fire and Forget" with full page reloads [29].	Event-Driven Control Loop; real-time state injection [29].
Visualization	Static coordinates; frequent node overlap [9].	Force-Directed Layout (Barnes-Hut simulation) [3].
Explainability	Abstract nodes only (Word1 $\rightarrow$ Word2) [29].	Contextual Modal Windows with Evidence Retrieval [29].

4.6 Advanced Visualization Modules

One last addition to enhance the **explainability** of the generated association rules, the system moves beyond standard graph layouts by incorporating two additional data representations: the **Adjacency Heatmap** and the **Chord Diagram**. These views provide alternative cognitive perspectives on the same underlying data structure ([GraphDt](#)), allowing users to uncover patterns that may be obscured in a *traditional force-directed graph*.

-
- **The Adjacency Heatmap: [10]** While the force-directed graph is excellent for showing **topology** (*clusters and hierarchy*), it often struggles to quantitatively compare the strength of individual relationships, especially in dense networks. To address this, an **Adjacency Heatmap** was implemented.
 - **Visual Encoding:** The Heatmap visualizes the association rules as an $N \times N$ matrix, where the rows represent *Antecedents (Parents)* and the columns represent *Consequents (Children)*.
 1. **Cell Color:** The saturation of each cell is directly mapped to the **Confidence** of the rule derived in the backend. A darker or more intense color indicates a *stronger probability of implication* ($P(Y|X)$).
 2. **Cell Ordering:** Rows and columns can be sorted by **Support** or **PageRank**, placing the most critical terms at the *top-left or center*.
 - **Contribution to Explainability:** The **Heatmap** creates a "Macro-View" of the mining results. It allows the user to instantly identify "*Hotspots*" of **Logic**—dense areas of the matrix where multiple terms strongly imply each other. This view eliminates the problem of **edge occlusion** (*lines crossing over each other*), making it easier to spot outliers (*weak links*) or dominant terms that drive the **taxonomy's structure**.
 - **The Chord Diagram: [12]** Is introduced to visualize the **inter-connectivity** and the "*flow*" of implication between concepts, which is particularly useful for identifying **cyclic relationships** or **mutual dependencies** that linear hierarchies might hide.
 - **Visual Encoding:** Concepts are arranged radially around a circle.
 1. **Arc Width:** The width of the connection (*the chord*) connecting two terms is proportional to the **Support** or **Lift** of the rule. Thicker chords represent statistically *significant*, frequently occurring relationships.
 2. **Node Arc Length:** The length of the arc on the circle's perimeter represents the node's **Weighted PageRank** or total degree, visually summarizing the *term's global importance*.
 - **Contribution to Explainability:** The **Chord Diagram** excels at revealing "*Bridge Concepts*". It allows the user to see terms that connect disparate semantic clusters (e.g., a term that links the "Political" cluster to the "Economic" cluster). Furthermore, it highlights the directionality of the rules; by tracing the width of the chord at the source versus the target, the user can visually assess the asymmetry of the relationship (*i.e., identifying if $A \rightarrow B$ is much stronger than $B \rightarrow A$*).

5 Data Description

To rigorously evaluate the proposed taxonomy generation engine—specifically assessing the efficacy of the **Eclat algorithm** for frequent itemset mining, the robustness of the **Weighted PageRank** centrality metrics, and the utility of the iterative **K-Edge pruning** strategies—we employed a stratified selection of text corpora. The evaluation utilized a diverse suite of datasets ranging from manually curated synthetic inputs to high-dimensional, unstructured real-world repositories. These datasets were categorized into three distinct tiers to test specific performance vectors of the system:

5.1 Corpus Used For Testing

1. **Verification and Calibration Corpora** The initial phase of evaluation focused on validating the logical correctness of the *association rule mining* engine (`Index.java`) and the filtering mechanisms (`POSCheck.java`). The dataset where:

- **SMcollection (Synthetic/Hand-Curated Validation):** A controlled, small-scale dataset manually designed to contain known latent structures. This corpus was utilized to verify the **Eclat algorithm's recursion** depth and to calibrate the **Binomial Coefficient thresholds** used for subset generation. By using a dataset with "ground truth" associations, we verified that the system correctly computed *Support and Confidence metrics* without introducing algorithmic noise.
- **AI-Generated Unstructured Text:** To assess the system's resilience to unstructured input lacking rigid formatting, we generated a synthetic corpus using a **Large Language Model (LLM)** with the prompt: "Can you create 40 AI-Generated movie descriptions of about 600 words each in txt form?". This dataset served to stress-test the `GranularityEnhancement` module, specifically evaluating the efficacy of the `semanticChunking` method (**Jaccard similarity**) in partitioning "free text" where standard sentence delimiters might be ambiguous.

2. **Standard NLP Benchmarks** To evaluate the system's performance under standard *Natural Language Processing (NLP)* conditions, we employed widely recognized benchmarks to test scalability and graph topology.

- **The 20 Newsgroups Dataset:** As the source explains it:
it contains 19997 articles for 20 categories taken from the **Usenet newsgroups collection**. We used the subject and the body of each message only. Some of the newsgroups are very closely related to each other (e.g., *IBM computer system hardware / Macintosh computer system hardware*), while others are highly unrelated (e.g. *misc forsale / social religion and christian*). This corpus is different from the previous corpora because it includes a larger vocabulary and words typically have more meanings. Moreover, the stylistic writing (*e-mail dialogues*) is very distant from the other more technical collections.

Source: [21]

This dataset was pivotal for evaluating the *Weighted PageRank* centrality metrics. We analyzed whether the algorithm could identify "hub" nodes (e.g., "CPU", "Government") that effectively cluster semantically related terms, distinguishing between polysemous words based on their graph neighbors.

- **Project Gutenberg Corpus:** A collection of classic literary works characterized by long-form narrative structures. We downloaded 1,200 free e-books using a custom made Python script and with some help from LLM we found the book IDs that we used. This corpus was used to test the `GranularityEnhancement` modes (`PARAGRAPH` vs. `SENTENCE`). It allowed us to analyze how changing the mining window size affects the density of the resulting taxonomy—specifically, whether broader paragraph-level windows capture thematic relationships better than strict sentence-level windows in narrative text.

Source: [26]

3. Domain-Specific and Multi-Format Corpora

The final phase evaluated the application's utility in specialized fields requiring precise terminology extraction and *multi-format data ingestion*.

- **MIMIC-III Clinical Database:** As the source explains it:
MIMIC-III is a large, freely-available database comprising deidentified health-related data associated with over 40,000 patients who stayed in critical care units of the Beth Israel Deaconess Medical Center between 2001 and 2012 [1]. The MIMIC-III Clinical Database is available on PhysioNet ([doi: 10.13026/C2XW26](https://doi.org/10.13026/C2XW26)). Though deidentified, MIMIC-III contains detailed information regarding the care of real patients, and as such requires credentialing before access. To allow researchers to ascertain whether the database is suitable for their work, we have manually curated a demo subset, which contains information for 100 patients also present in the MIMIC-III Clinical Database. Notably, the demo dataset does not include free-text notes.

Source: [19]

This dataset tested the **Part-of-Speech (POS) Tagger's** (`POSCheck.java`) ability to filter meaningful medical nouns from administrative jargon. Furthermore, it evaluated the Lift metric (`Rule.java`) in identifying rare but critical clinical associations (e.g., *symptom* → *diagnosis*) that might be overshadowed by common *stopwords* in a standard frequency analysis.

- **Scientific Literature Collection:** Our biggest dataset, containing PDF files summing up to 2.42GBs. A corpus of academic papers in PDF format. This dataset was specifically selected to validate the backend's PDF ingestion pipeline (`VisualRestController.java`), which utilizes `PDFTextStripper` with coordinate-based sorting (`setSortByPosition(true)`).

This test case ensured that the system could correctly linearize multi-column academic layouts before processing, a critical step for preserving the semantic integrity of the generated rules.

Table 7: Stratified Selection of Text Corpora for System Evaluation

Tier	Dataset(s)	Primary Evaluation Objective
1. Verification	SMcollection, AI-Generated Text	Validating Eclat recursion depth and Jaccard similarity efficacy.
2. Benchmarks	20 Newsgroups, Project Gutenberg	Identifying "hub" nodes via PageRank and testing paragraph vs. sentence granularity.
3. Domain-Specific	MIMIC-III, Scientific PDFs (2.42GB)	Testing POS filtering of medical jargon and PDF coordinate-based linearization.

6 Design and Modeling

6.1 Domain Model And Data Structures

Unlike traditional web applications that rely on persistent relational databases (*SQL*) to store entity relationships, this system operates on a high-performance **In-Memory Object Model**. Given the ephemeral nature of exploratory text mining—where a user might generate thousands of **association rules** in seconds and then discard them just as quickly—persisting every intermediate state to a *disk-based database* would introduce *unacceptable I/O latency*.

Instead, the system models the domain using **strict Java Object-Oriented** structures that reside in the application heap. This "*Transient Graph Model*" is defined by three core classes.

6.1.1 The Graph Data Transfer Object

The `GraphDt` class functions as the primary **Data Transfer Object (DTO)**, a design pattern used to aggregate data for network transmission. Its architecture is optimized for efficient serialization into **JSON** format, serving as the *interface contract* between the computational backend and the visualization frontend.

- **Structure:** Unlike a normalized database schema, the class utilizes a denormalized, **document-oriented structure**. It encapsulates the taxonomy into **two** distinct collections:
- **Nodes Map:** A `Map<String, Map<String, Object> >` structure is employed rather than a simple list. This **key-value mapping** (*where the Key is the term ID*) reduces lookup complexity from $O(n)$ to $O(1)$ during the frontend rendering phase.

- **Edges List:** A `List<Edge>` collection represents the **directional relationships**. These are transient entities, *dynamically instantiated based on the user's active filtration thresholds*.
- **Performance Metrics:** The class extends beyond data storage to include execution metadata, specifically the **time** attribute. This allows the frontend to act as a profiler, *providing the user with real-time feedback on the computational cost of the mining algorithms*.
- **Simple JSON Example:**

```

1 {
2   "time": 63,
3   "nodes": {
4     "isnt": {
5       "level": 1,
6       "rank": 0.04977699850249309,
7       "label": "isnt"
8     },
9     "original": {
10      "level": 1,
11      "rank": 0.04179624252905502,
12      "label": "original"
13    },
14  },
15  "edges": [
16    {
17      "from": "isnt",
18      "to": "perfect",
19      "support": 0.38095238095238093,
20      "confidence": 0.8888888888888888
21    },
22  ]

```

Listing 6.1: Real Example JSON Payload

6.1.2 The Semantic Node Entity

The `Node` class models the vertices of the semantic graph. It is designed as a **Doubly-Linked Data Structure**, maintaining references to both its conceptual *antecedents* (**parents**) and *consequents* (**children**).

- **Topological Awareness:** By explicitly storing `List<Node>` parents and `Map<Node, List<Double> >` children, the entity enables efficient **graph traversal**. This bidirectional linking is a prerequisite for the system's "Hierarchy Leveling" heuristic, which calculates a node's vertical depth based on the ratio of its **In-Degree to Out-Degree**.
- **Weighted Connectivity:** The **children** map stores not just the **target** node, but a `List<Double>` representing the edge **weights** (*Confidence and Support*). This ensures that the graph topology inherently carries the statistical strength of the underlying association rules.

6.1.3 The Association Rule Abstraction

The system distinguishes between the **mathematical** concept of a "*Rule*" and the **visual** concept of an "*Edge*". Moreover:

- **The Rule Model:** The `Rule` class encapsulates the statistical rigor of the **Eclat algorithm**. It acts as an immutable record of a discovered pattern, storing the three critical metrics: **Support** (*frequency magnitude*), **Confidence** (*conditional probability*), and **Lift** (*correlation strength*).
- **The Edge Model:** The `Edge` class is the visual projection of a `Rule`. It explicitly decouples the source (`from`) and target (`to`) identifiers to satisfy the input requirements of the frontend's **Force-Directed Layout engine**. The transformation from `Rule` to `Edge` occurs dynamically in the `Controller`, acting as a filtering layer that only instantiates edges meeting the user's current confidence criteria.

6.2 Application State Modeling

A significant challenge in **RESTful** architectures is the stateless nature of `HTTP`. However, exploratory data analysis requires "*Session Context*"—the system must retain the mined rules and the loaded corpus to allow for **rapid, iterative filtering without reloading the raw data**.

6.2.1 The Static Singleton State Pattern

To resolve this, the `VisualRestController` implements a **Static State Model**, effectively functioning as a **Singleton State Machine**

- **In-Memory Persistence:** Critical simulation variables—specifically the `index` (the inverted index of the corpus) and rules (the `HashMap` of mined associations)—are declared as **private static fields**.
- **Architectural Implication:** This design decision trades horizontal scalability for vertical performance. By keeping the `Index` object in static memory, the system avoids the expensive overhead of re-tokenizing and re-indexing the text corpus every time the user adjusts a slider. When a parameter changes, the system simply accesses the pre-computed index and re-runs the lightweight rule filtering logic (`newProcess`), delivering **near-instantaneous feedback**.

6.3 RESTful Interaction Modeling

The interaction between the **Client** (Browser) and **Server** (Java) is formalized through *a Strict Contract*, eliminating ambiguity in data exchange and ensuring type safety.

6.3.1 The Input Contract

`ValueRequest` All state-changing operations are encapsulated within the `ValueRequest` class, a **Plain Old Java Object (POJO)** designed to bind incoming **JSON** payloads.

- **Decoupling:** This class decouples the API endpoint signature from the internal logic. It aggregates diverse parameters—File System paths (*path*), algorithmic constraints (*phrase-length*), and statistical thresholds (*support*, *confidence*)—into a single composite object. This ensures that the `VisualRestController` remains resilient to future parameter additions.

6.4 User Interface Design Patterns

The frontend architecture shifts from a linear document model to a **Component-Based Dashboard**, implementing specific **UX** patterns to handle information density.

6.4.1 The Split-Pane Control Pattern

The layout implements a separation of concerns between the "*Control Plane*" and the "*Data Plane*". Expanding on this we have:

- **The Control Sidebar:** Modeled as a fixed-position view that can disappear whenever the user wants to check in more detail the **Topology**. This component manages the input state. It allows for "*Simultaneous Tuning*", enabling users to manipulate variables (*Support/ Confidence/ Lift/ KNodes/ Font Size/ Search Node/ Controls*) while maintaining visual focus on the resulting changes in the **graph Topology**.
- **The Visualization Canvas:** A fluid, Z-indexed container designed to layer multiple visualization engines (**Vis.js Network**, **D3.js Chord**, **D3.js Heatmap**) within a single spatial context. This allows the user to view the created **Topology** in different ways.

6.4.2 The Asynchronous Event-Driven Loop

The application interaction is modeled on an **asynchronous Event Loop**, distinct from the synchronous request-response cycle.

- **Simulation vs. Execution:** Upon submission, the **UI** triggers a "*Timed Simulation*" sequence. This logic mirrors the internal phases of the backend (*e.g.*, "*Mining Rules*," "*Calculating PageRank*") in the **UI**, bridging the psychological gap caused by backend latency and at the same time it gives the user a "*peek*" at what step the process is on the backend.
- **State Injection:** The *JavaScript* logic captures the form state, serializes it into the `ValueRequest` structure, and injects it into the backend via a fetch **API call**. The subsequent rendering phase is strictly data-driven, rebuilding the **DOM** solely based on the received `GraphDt` state, ensuring the visualization is always a pure reflection of the underlying mathematical model.

6.5 Diagrams

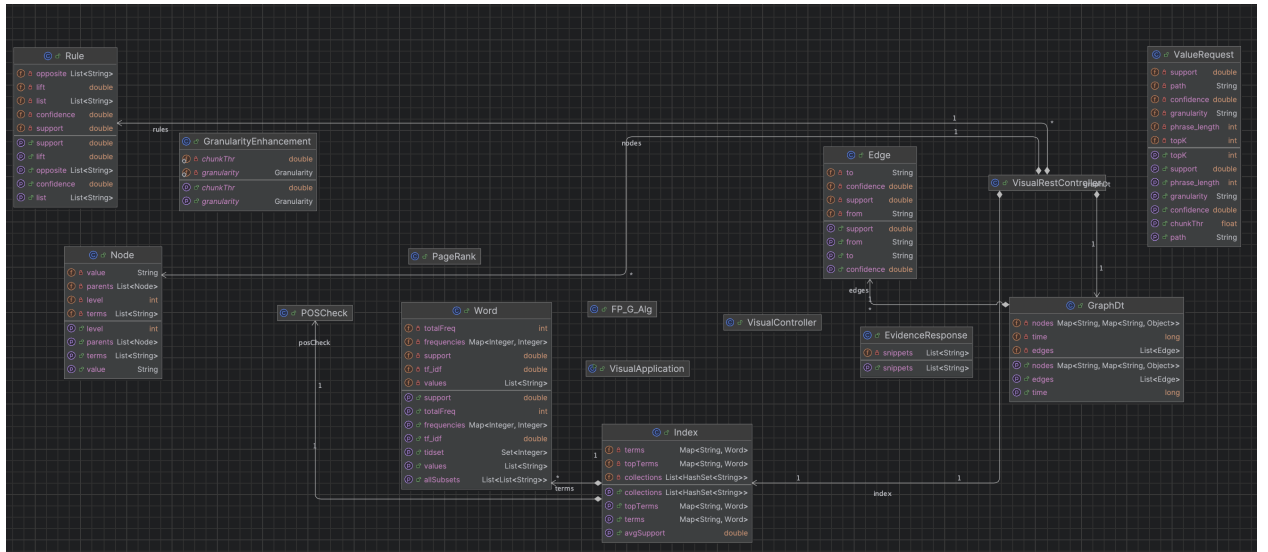


Figure 1: This figure illustrates the **system's class architecture**, highlighting the separation between computational logic and data representation. At the core, the **Index** class aggregates the corpus into **Word** objects using the **GranularityEnhancement** strategy for segmentation. The mining results are transformed into a transient graph model via the **GraphDt** class, which encapsulates the topology using **Node** and **Edge** objects. Finally, the **VisualRestController** orchestrates the flow, accepting user parameters via **ValueRequest** and returning the structured taxonomy, ensuring a strict contract between the backend mining engine and the frontend visualization layer.

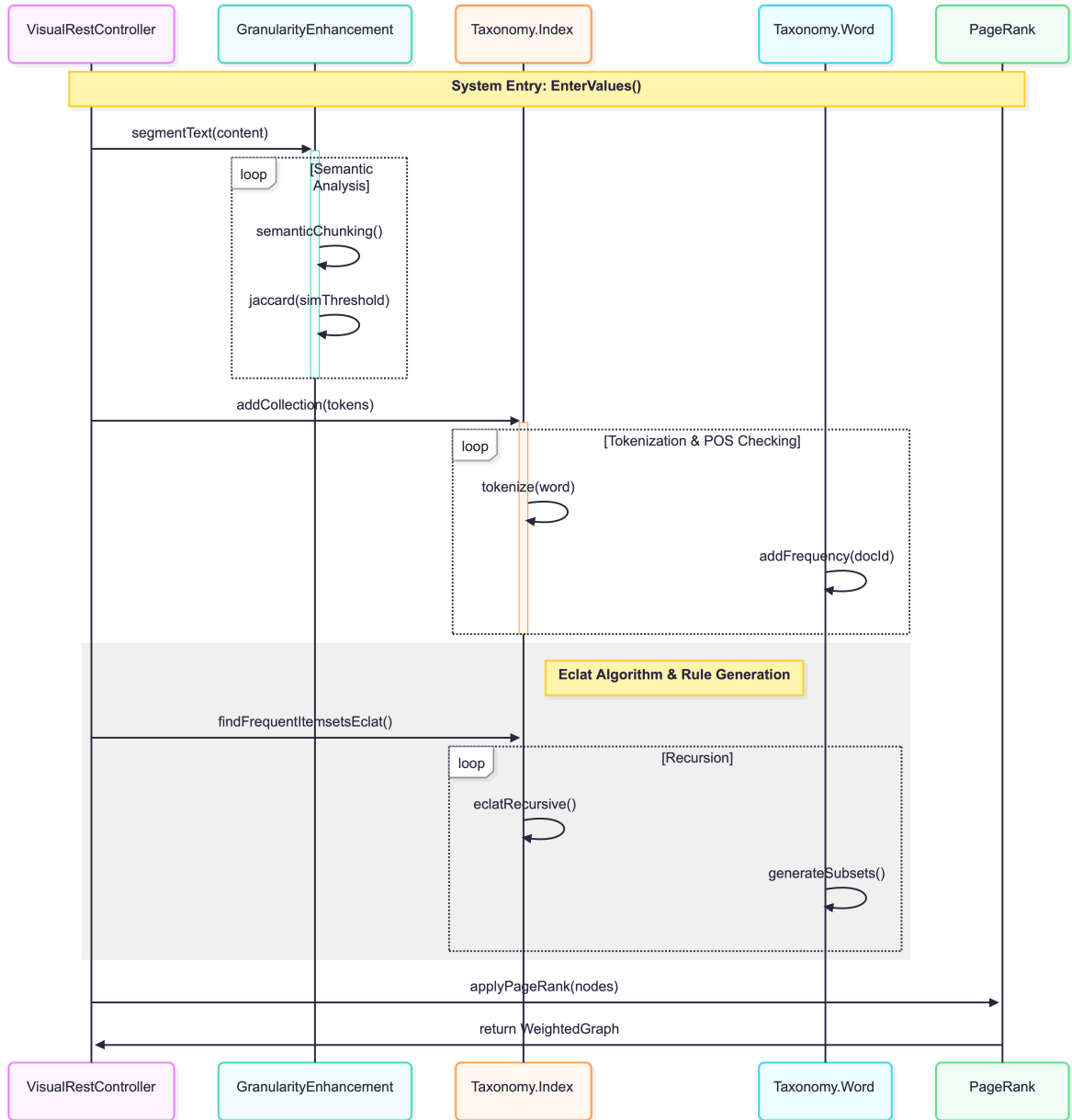


Figure 2: The sequence diagram illustrates the synchronous execution pipeline initiated by the `EnterValues` endpoint in the `VisualRestController`. The process begins with **Data Preparation**, where the `GranularityEnhancement` module dynamically segments the raw text—executing **iterative Jaccard similarity checks** if semantic chunking is active—before passing the tokens to the Index for normalization and frequency counting. The workflow then transitions to the **Mining Phase**, where the `findFrequentItemsetsEclat` method performs a **recursive Depth-First Search (DFS)** to identify frequent itemsets via transaction ID intersection. Finally, the **Topology Phase** invokes the `PageRank` algorithm to iteratively calculate node centrality weights, constructing and returning the final `WeightedGraph` to the controller.

7 System Architecture and Tools

7.1 Architecture and Design of the REST API

While the previous chapter defined the *abstract data models*, this section details **the concrete software architecture and the technological stack** used to implement the backend services. The system is built as a *monolithic Java application*, orchestrated by the **Spring Boot** framework, which provides the embedded servlet container (Tomcat) necessary to serve the **REST** endpoints.

7.1.1 Files and Structure

The backend code is organized into a modular package structure under the `org.example` namespace, enforcing a clear separation between the **web interface** and the **core mining logic**. The project follows the standard Maven directory layout:

- **View Package** `org.example.View`: This acts like the **API Layer**, moreover:
 - `VisualApplication.java`: The entry point containing the `main` method, annotated with `@SpringBootApplication` to bootstrap the Spring context.
 - `VisualRestController.java`: The primary controller that handles **HTTP POST** requests. It does not contain business logic itself but acts as a facade, **routing** user inputs to the appropriate mining services.
 - `GraphDt.java` & `Edge.java`: These classes are located here because they are strictly coupled to the **JSON** response format required by the frontend, utilizing Jackson annotations for serialization.
- **Taxonomy Package** `org.example.Taxonomy`: This acts like the **Service Layer**
 - `Index.java`: **The computational engine**. It encapsulates the file reading, tokenization, and the recursive Eclat algorithm implementation.
 - `POSCheck.java`: An isolation layer for the *Natural Language Processing* dependencies, ensuring that the complex **Stanford CoreNLP** initialization is contained within a single utility class. This class was not changed in our work, it was made and maintained as it is from E. Mitkousis in his original work.

7.1.2 Connection and Data Flow

The execution flow is synchronous and request-driven. When the `VisualRestController` receives a payload, it triggers a linear pipeline, looking like:

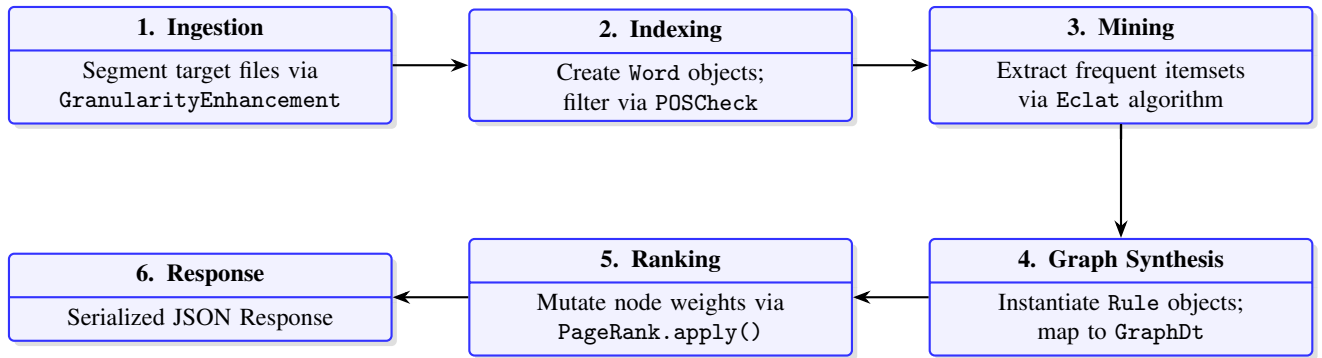


Figure 3: The transformation from raw file ingestion to the final ranked graph structure, following in the steps of the system itself.

7.1.3 Technologies And Tools

- **Spring Boot (v3.x):** Chosen for its *"Convention over Configuration"* approach, allowing for rapid deployment of the **REST API** without complex **XML** configuration.
- **Stanford CoreNLP:** Utilized specifically for its **Maximum Entropy Part-of-Speech (POS)** tagger, which provides higher accuracy than rule-based stemmers for distinguishing nouns and adjectives.
- **Jackson (fasterxml):** The **JSON** processor used to marshall the Java Object Model into JavaScript-compatible payloads. It is critical for handling the circular references in the graph structure (*Nodes referencing Parents*).
- **Lombok:** utilized to reduce boilerplate code (*getters/setters/loggers*) in the controller classes.

7.2 Design and Modeling of the Web Application

The frontend architecture serves as the presentation layer, consuming the **JSON** output from the **REST API**. Unlike the backend, which is structured across multiple Java packages, the frontend is implemented as a **lightweight Single Page Application (SPA)** embedded directly within the Spring Boot resource path. In conclusion, the frontend architectural design **was not changed**, it remained as a **SPA**, the core code of it on the other side is rewritten to a new more *user friendly & interactive* application providing the user with more filter options and different views for their **Taxonomy**. In more detail the frontend looks like:

7.2.1 Files and Structure

The visualization logic is contained entirely within `visualize.html`. This approach is following the structure of *E. Mitkousis* in his original work. The above file is **separated** into 3 distinct pieces of code, each with its own importance degree:

Styles code/CSS: Located in the beginning of the `visualize.html`, inside the `<style>` block, defining the responsive grid variables and the z-index layering for the modal windows. **Responsible**, for all the styling of the frontend page.

Structure code/HTML: Located right after the end of **CSS**, Defines the split-pane layout, containing the `#sidebar` for configuration and the `#network-container` for the canvas. **Responsible**, for all the structure of the page - where everything is located.

Functions/JavaScript Code: Located after the end of the **HTML**, the largest component, containing the event listeners for the *"Submit"* button, the **fetch()** logic for API communication, and the **rendering functions** for the visualization libraries.

- **Vis.js (Network Module):** Chosen for its high-performance physics engine (Barnes-Hut solver). It handles the *Force-Directed Graph rendering*, allowing for interactive dragging, zooming, and clustering of thousands of nodes.
- **D3.js (Data-Driven Documents):** Utilized for the advanced *"Macro-Views"* (*Chord Diagram and Heatmap*). D3 was selected for its ability to bind data directly to SVG elements, enabling the complex arc calculations required for the Chord layout and the dynamic color scaling of the Heatmap.

8 UI Interface Update

Having established the underlying architectural changes, this section shifts focus to the user-facing elements by providing a visual *comparison* of the interface before and after the redesign. The purpose of this visual retrospective is to quantify the application's aesthetic and functional evolution. It serves as visual proof that the development process successfully adhered to its initial mandate: to provide users with a **cleaner, more informative, and explanatory** platform for text analysis.

8.1 New UI Style: Frontpage and New Dashboard Bar

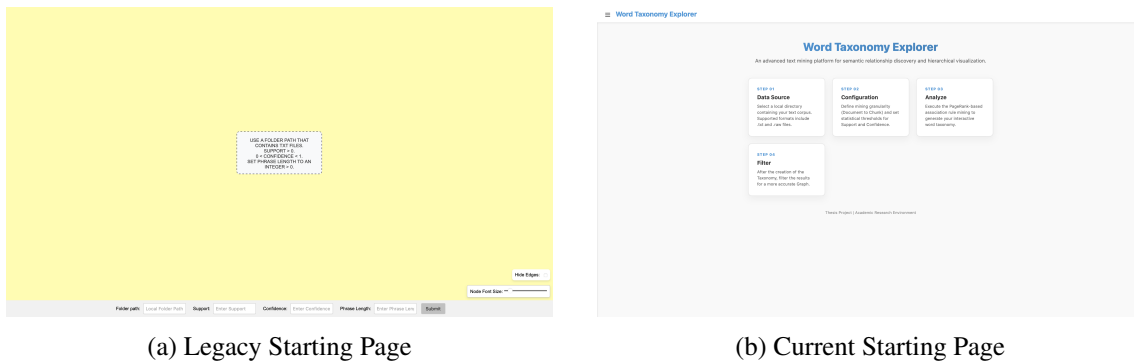


Figure 4: A visual comparison between the legacy linear layout and the current more user friendly "first look" of the application.

Another **addition** to our application is the *Interactive New Dashboard*, from where the user can apply the new implemented filters that work on the backend. In context:



Figure 5: The legacy *Control Panel* with all its options and View settings.

PARAMETERS

Folder path:

SMcollection

Select

Text Mining Granularity:

Document

Support:

0.9

Confidence:

0.9

Chunk Threshold:

Enter Threshold (if CHUNK)

Phrase Length:

1

Top-K Rules (by PageRank):

100

Submit

(a) Current Parameters Setting

VIEW MODE

Switch between visualization types.

Network View

Chord View

Heatmap

(b) Current View Mode

VIEW FILTERS

💡

Use these sliders to **instantly filter** the graph you see.
To discover new rules, change the 'Parameters' at the top and click 'Submit' again.

Min. Confidence: 0.5

Min. Lift: 1.0

Min. Support: 1.0

Max Edges per Node (K):

e.g. 2 (Empty = All)

Limits connections per node to reduce clutter.

Apply Filters

(c) Current Filters Panel

NETWORK EXPLORATION

Search Node

Enter node label...

Search

View Controls

Ego View

Reset View

Cluster Graph

Un-Cluster All

VIEW OPTIONS

Node Font Size:

Hide Edges

(d) Current Network Exploration Panel

Figure 6: A comprehensive visual representation of the current *Dashboard* from where the user CSD can now control new filters, as well as view their *KG* in other ways too.



8.2 Graph Style

In this section we will look at how the two applications showcased the same **Data Collection**, with the same settings: for the Granularity field i used the *Document* option since the legacy application only worked with that setting.

Support: 0.35

Confidence: 0.35

Sentence Length: 1

Granularity: Document

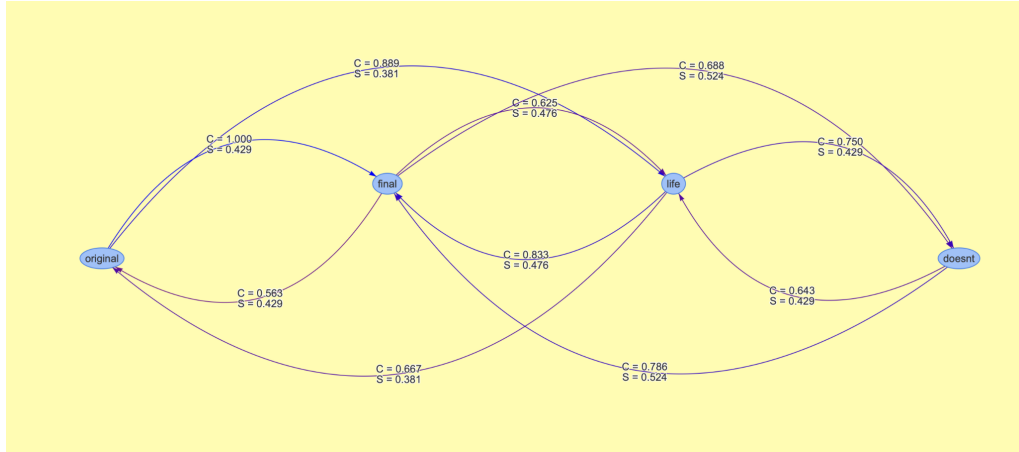


Figure 7: The legacy result Graph with our visual testing settings.

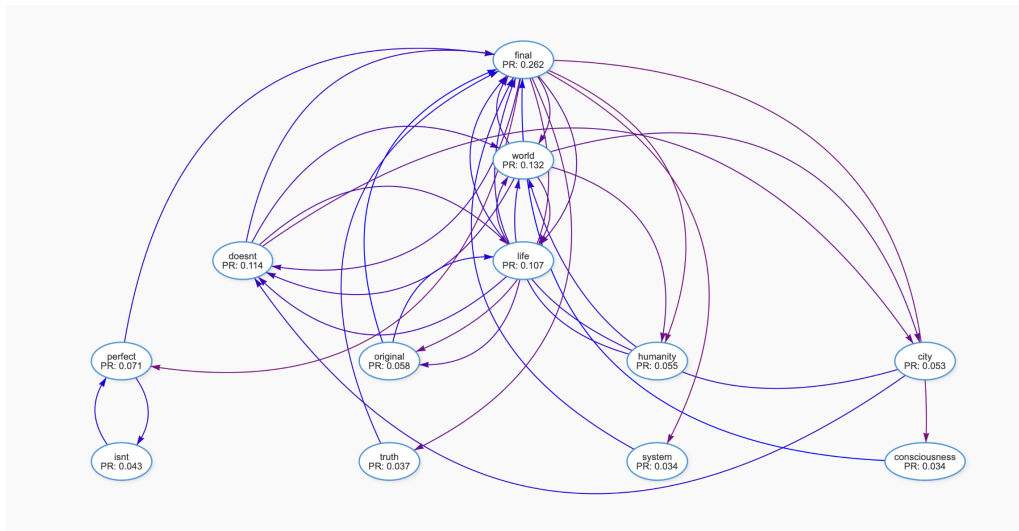


Figure 8: The current result Graph with our visual testing settings.

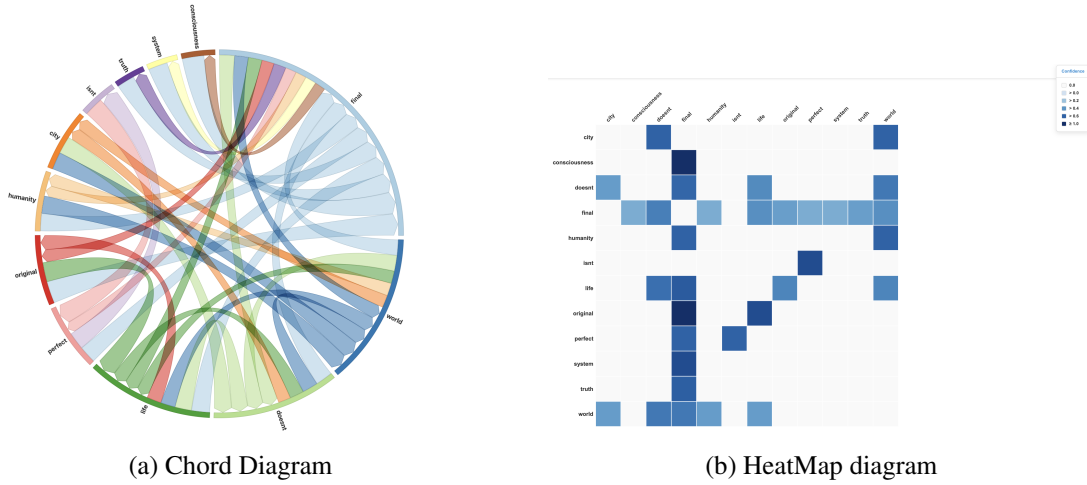


Figure 9: Here, we can observe the newly supported visualization modes, which provide the user with diverse ways to interpret and interact with the extracted data.

8.3 Graph Details On-The-Spot Analysis

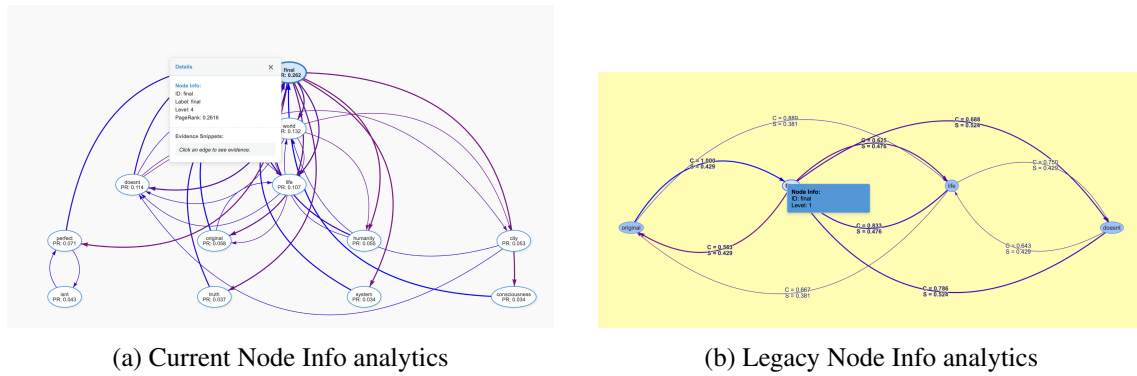


Figure 10: The different information display when a user clicks on a Node inside the Graph.

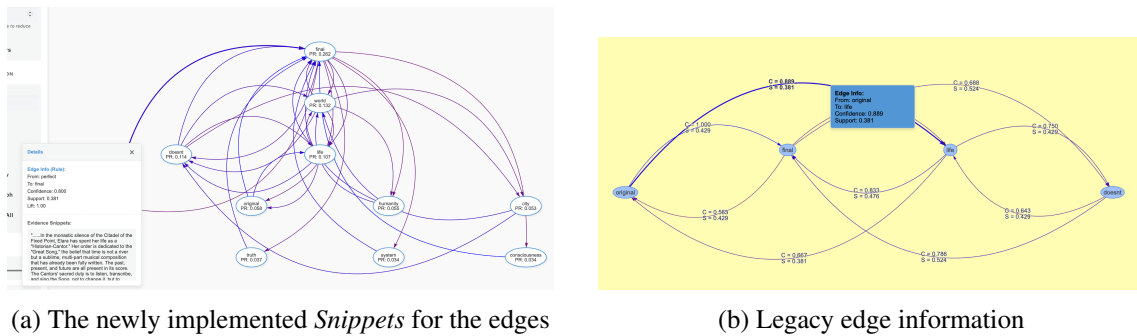


Figure 11: Here, we can observe the newly supported *Snippets* for on the spot Edge analysis.

Another visual addition is the **Simulated Loading State**. When the user clicks "Submit", the UI obscures the canvas with a frosted-glass overlay and displays a multi-stage progress bar. Instead of a generic spinner, the UI provides textual updates corresponding to **the backend's current task** (e.g., "Reading Corpus...", "Mining Association Rules...", "Ranking Nodes..."). This provides vital psychological feedback during computationally heavy operations.

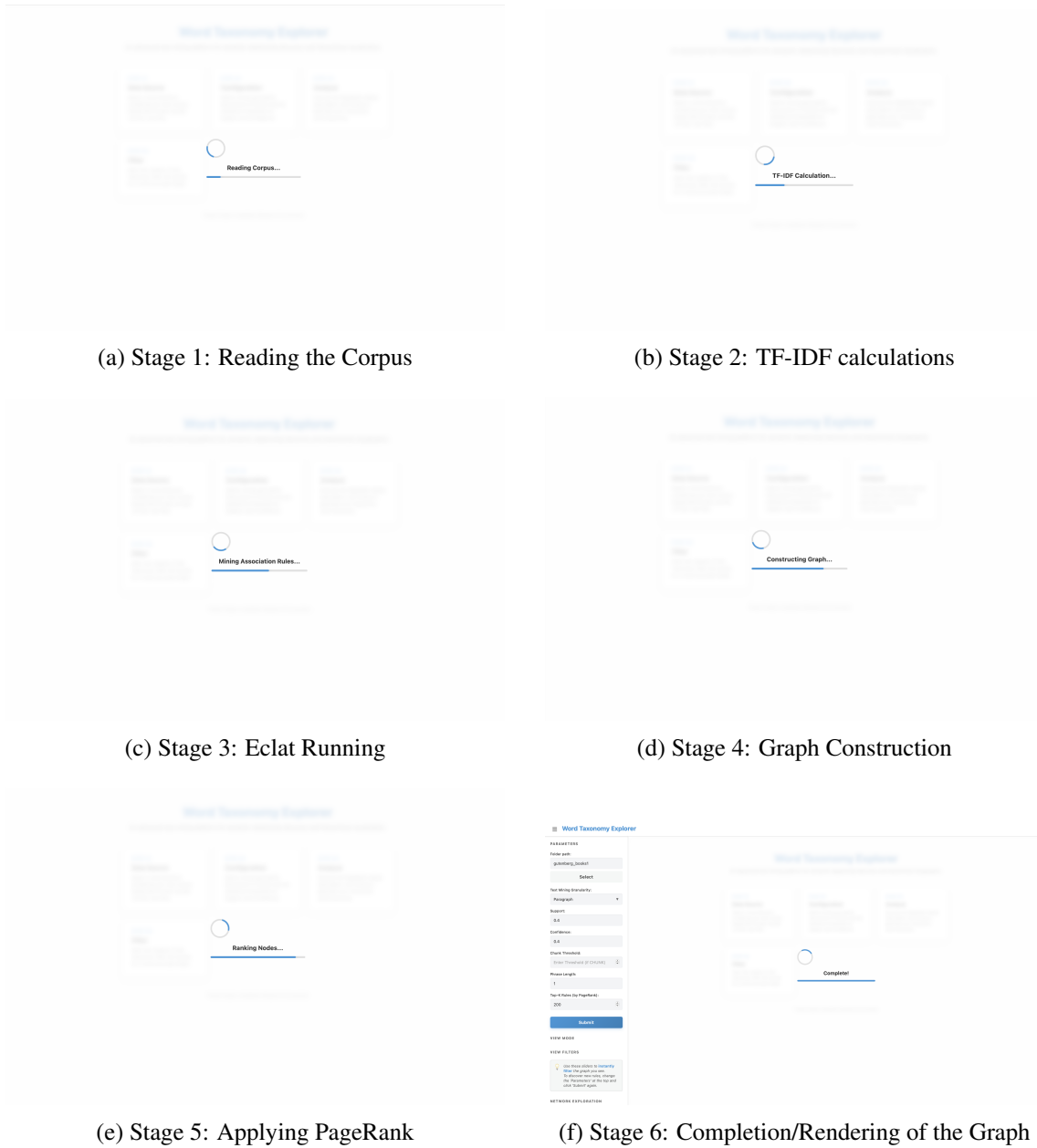


Figure 12: A sequential visual breakdown of the application's loading states and interface components. The progression illustrates the real-time feedback mechanisms now available to the user.

8.4 Live Filtering

With the same **Data collection**, and the same *Confidence and Support* settings we will also take a look at how the new **Live Filtering** options apply on the Graph, and if they actually make the data more robust and easy for the user to read:

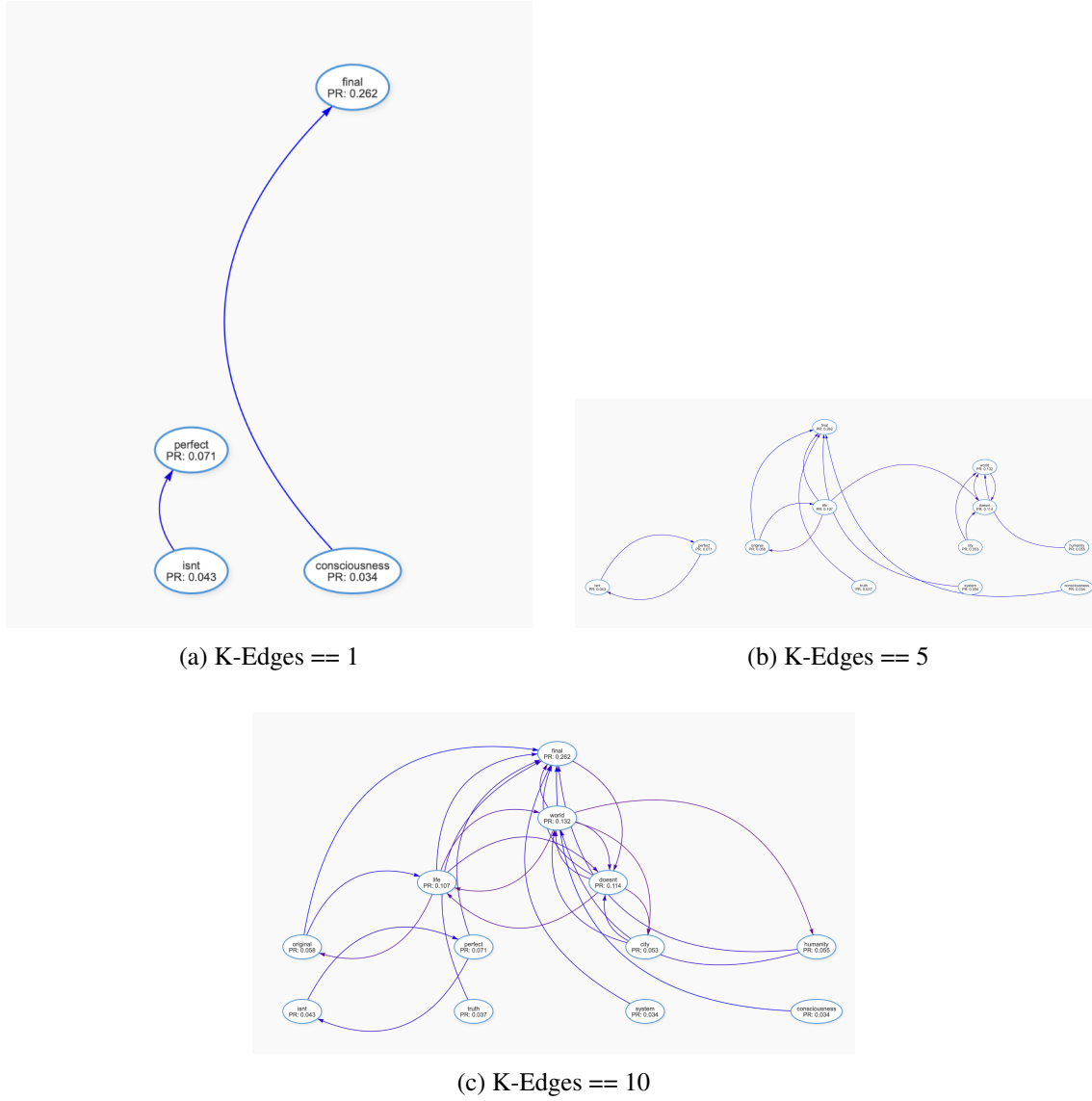
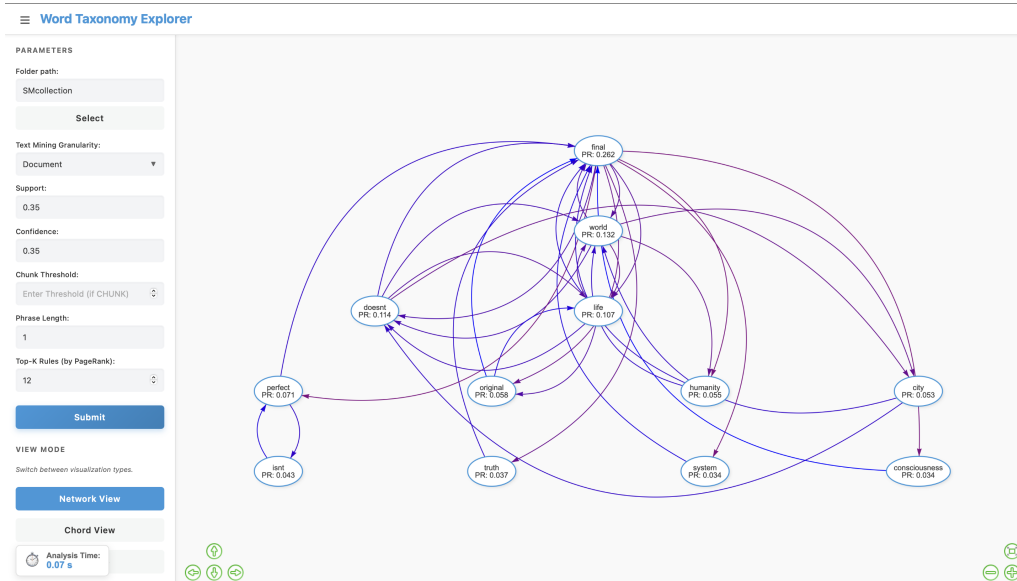


Figure 13: A visual overview of the three distinct analysis modules after live changing the number of edges that we want displayed.

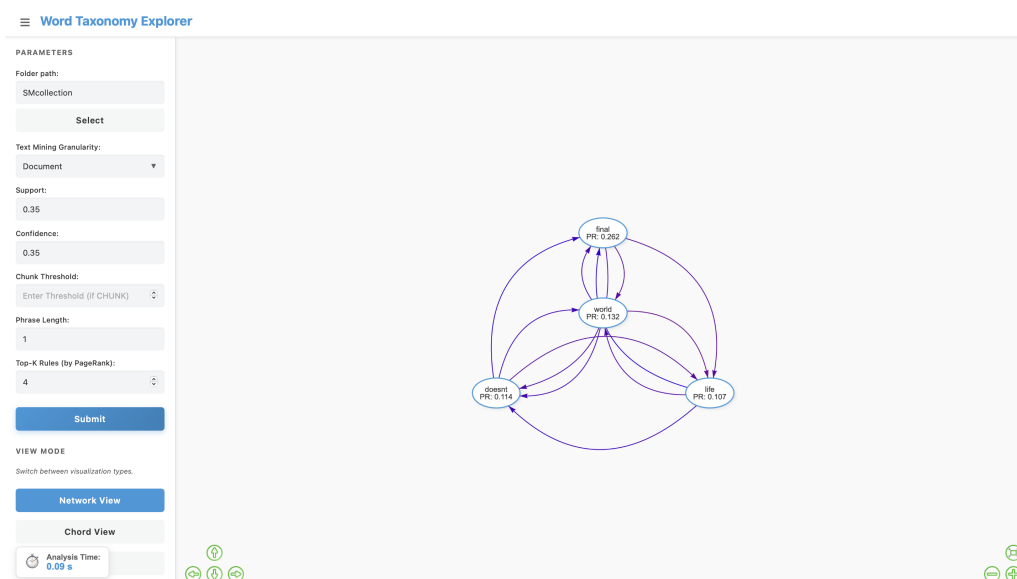
- **Conclusion:** A comparative analysis between these filtered representations and the initial complete taxonomy (Figure 8) reveals a substantial improvement in visual clarity. Because the backend algorithm systematically retains only the edges with the highest statistical weight, reducing the **K-Edge** parameter acts as a strict pruning mechanism. While this inevitably results in the loss of secondary contextual information, it successfully isolates the most robust semantic associations, providing a highly distilled view of the core conceptual relationships.

An additional, and arguably, *interesting way* to examine our newest "*Live Filtering*" option is: **Setting the *Top-K Rules* parameter directly to 12, and then observing the resulting taxonomy and compare it to the one we get when we use the "*Live Filtering*" option with *K-Edges* == 10.**



The resulting taxonomy is **identical** to our original one (Figure 8). The reason for this result is: **The option K-Edges == 10 is the pinpoint value where it gets all of 12 nodes of the taxonomy involved.** Meaning, with 10 edges the taxonomy can *at the same time show all of the created rules and let us explore the "strongest" ones, by showcasing the the edges with the highest confidence/support/lift metrics.*

One more interesting testing opportunity worth exploring is setting the **Top-K Rules** value == 4, to observe if the resulting taxonomy resembles the one on Figure 13a:



Attempting to replicate the output of the K-Edge filter by simply restricting the PageRank Top-K nodes yields mismatched results because *the metrics measure two different phenomena:*

Global semantic importance: The graph is reduced **by selecting the Top-K nodes based on their PageRank score**, the visualization highlights the broad "hubs" of the corpus.

Conditional syntactic probability: The graph is subjected to iterative K-Edge filtering, the topology shifts dramatically. **Because the algorithm prioritizes the retention of edges with the highest Confidence score.**

9 System Testing

9.1 Web Application Testing

In this section, we will conduct a side-by-side evaluation of the updated **Word Taxonomy Explorer** and its **predecessor**. Our objective with this *system testing* is to evaluate the software’s operational integrity by *focusing on two primary domains*. **First**, the computational efficiency of the redesigned backend algorithms. **Second**, the interactive reliability of the frontend dashboard. The foundational question this experimental chapter seeks to answer is whether the architectural upgrades have empirically and successfully mitigated the limitations of the legacy system.

9.1.1 Preliminaries

To ensure a rigorous and equitable comparative analysis, both the legacy and current iterations of the Word Taxonomy Explorer were subjected to identical workloads. The tests utilized the standardized corpora detailed in Chapter 5. Performance metrics were systematically recorded to evaluate computational efficiency, algorithmic scalability, and resource utilization.

Given the memory-intensive nature of high-dimensional association rule mining—particularly the legacy Apriori algorithm’s well-documented susceptibility to `OutOfMemoryError` (Java Heap Space) exceptions during candidate generation—the evaluation was conducted across two distinct hardware environments. This dual-system approach allowed us to benchmark baseline performance on standard consumer hardware, while also testing maximum scalability on a high-performance workstation.

The hardware and software specifications for the testing environments are as follows:

- **Environment A (Mobile/Baseline):** Apple MacBook Air featuring an M1 Silicon chip (8-core CPU), 8 GB of Unified RAM, running macOS Sequoia.
- **Environment B (High-Performance Workstation):** A custom-built desktop computer equipped with an Intel Core i9-12900K processor, 32 GB of DDR5 RAM, and an NVIDIA RTX 4070 Graphics Processing Unit (GPU), running Windows 11.

To standardize the reporting of the experimental results across both hardware environments and algorithms, the data was collected and structured using the reference format outlined in the following Table.

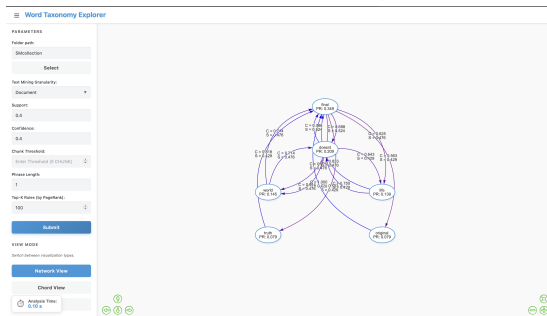
Dataset	Algorithm	Supp. (%)	Conf. (%)	Algo Time (s)	Total Time (s)	Memory Status
SMcollection (64KB)	Apriori	x%	y%	[A T]	[T T]	z%
	Eclat	x%	y%	[A T]	[T T]	z%
20 News Groups (40MB)	Apriori	x%	y%	[A T]	[T T]	z%
	Eclat	x%	y%	[A T]	[T T]	z%
MIMIC-III (100MB)	Apriori	x%	y%	[A T]	[T T]	z%
	Eclat	x%	y%	[A T]	[T T]	z%
Gutenberg Books (500MB)	Apriori	x%	y%	[A T]	[T T]	z%
	Eclat	x%	y%	[A T]	[T T]	z%
PDF Corpus (2.4GB)	Apriori	x%	y%	[A T]	[T T]	z%
	Eclat	x%	y%	[A T]	[T T]	z%

Table 8: Expanded performance metrics comparing the legacy and current algorithms. This table isolates the raw algorithmic execution time from the total system processing time (which includes file ingestion and NLP tokenization).

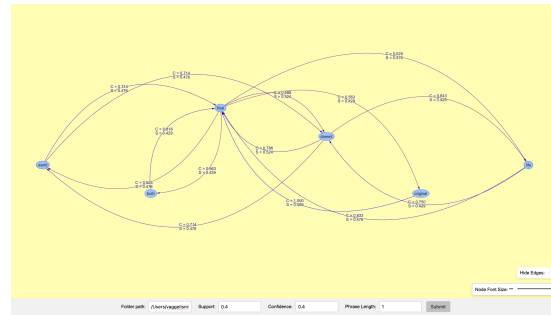
9.2 System Experiments on macOS

We began testing both systems on my personal laptop computer. The objective of these first experiments was to actually observe how much did the App become more user friendly: meaning *minimazing the need of a high end PC set-up* to calculate, visualize and explore a **Taxonomy**.

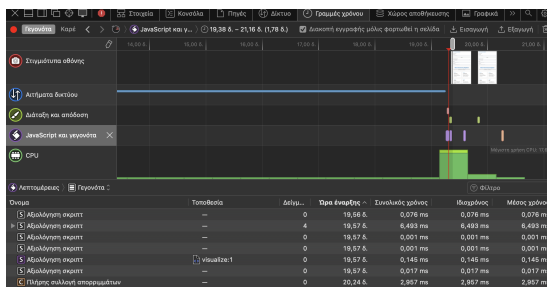
9.2.1 SMCollection



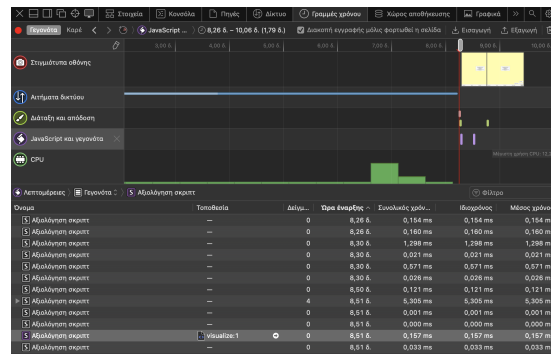
(a) Current Graph



(b) Legacy Graph

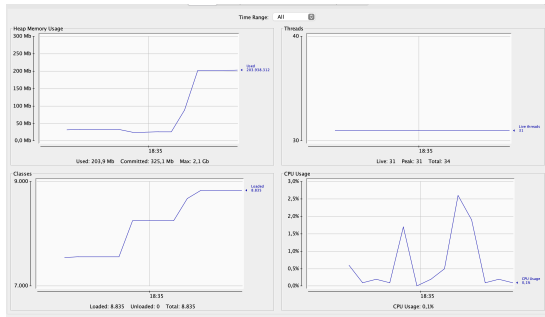


(c) Metrics for Eclat from Browser

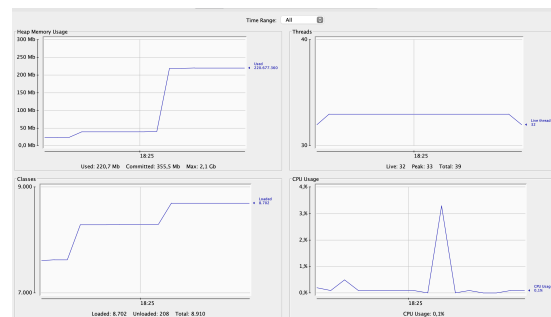


(d) Metrics for Apriori from Browser

After getting metrics from the browser we needed to actually observe the real-time **CPU** as well as **Memory** usage of each App. I used the `jconsole` command after running the same **Dataset** on both explorers which led to the following results:



(a) Eclat (new)



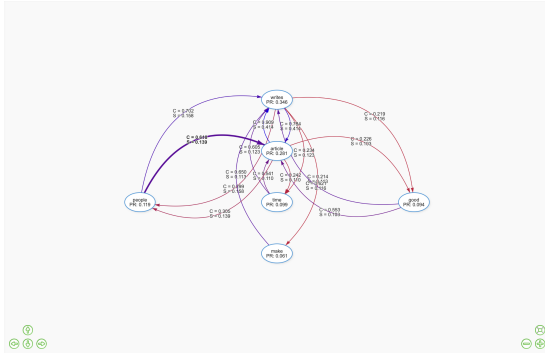
(b) Apriori (Legacy)

While both execution profiles exhibit the **expected correlation** between class loading and heap memory allocation, the Eclat implementation *demonstrates improved resource efficiency*. By leveraging a depth-first search and vertical data format, **Eclat mitigated the high-intensity CPU spikes and candidate-generation memory overhead characteristic of Apriori's breadth-first approach**. Consequently, the system achieved a lower peak heap memory footprint (*203.9 MB compared to 220.7 MB*) and maintained a strictly constant thread pool.

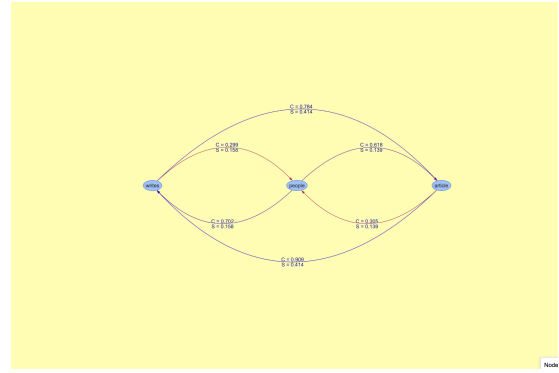
Dataset	Algorithm	Supp.	Conf.	Algo Time	Total Time	Memory Status
SMcollection (64KB)	Apriori	40%	40%	5ms	157ms	220,7Mb - 3.2%
	Eclat	40%	40%	3ms	145ms	203,9Mb - 2.6%

Table 9: We can instantly see that in a small Dataset even when the updated App has to run more processes it still manages to run faster than the legacy Apriori approach

9.2.2 20 News Groups Collection



(a) Current Graph



(b) Legacy Graph

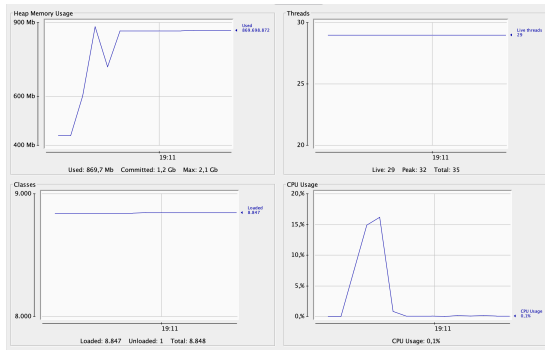
```
[TIME] Rule generation (Eclat): 16 ms
[TIME] Graph construction (nodes/edges): 1 ms
[TIME] GraphDt population: 0 ms
Applying PageRank...
[TIME] PageRank: 0 ms
[TIME] TOTAL execution time: 10815 ms
```

(c) Metrics for Eclat from Java Library

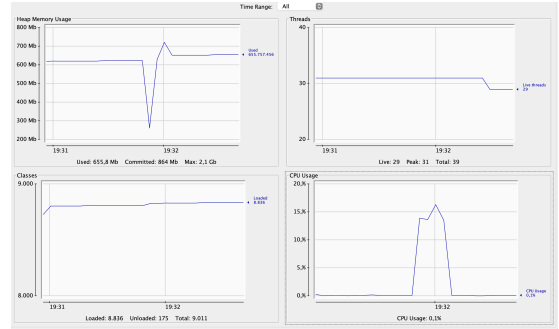
```
Apriori execution took: 486 ms
DONE FOR: 19998 FILES
--- TIMING REPORT ---
Total Request: 15580 ms
-----
```

(d) Metrics for Apriori from Java Library

Following the same logical steps as we did in 9.2.1 the results that we get are:



(a) Eclat (new)



(b) Apriori (Legacy)

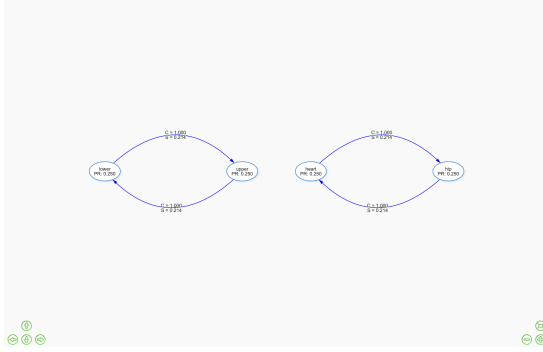
Testing the algorithms against a larger dataset reveals a nuanced performance **trade-off** between Eclat and Apriori. While Eclat previously demonstrated absolute resource efficiency on smaller workloads, the scaled dataset **highlights the memory-intensive nature of its vertical data format**. Consequently, Eclat required a higher sustained heap memory footprint (869.7 MB) compared to Apriori (655.8 MB).

However, Apriori's execution profile exposes a major Garbage Collection pause immediately preceding its memory peak, indicative of the severe object churn caused by its breadth-first candidate generation. Eclat, by contrast, followed a direct memory allocation path completely free of major GC interruptions and maintained absolute thread stability.

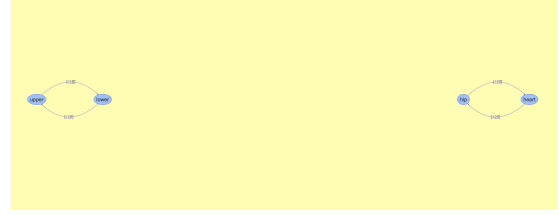
Dataset	Algorithm	Supp.	Conf.	Algo Time	Total Time	Memory Status
20 News Groups (40MB)	Legacy (Apriori)	10%	10%	486ms	15580ms	655.8Mb - 16%
	Current (Eclat)	10%	10%	16ms	10815ms	869.7Mb - 16%

Table 10: Significantly lower Total Time with a Memory Cost

9.2.3 MIMIC-III



(a) Current Graph



(b) Legacy Graph

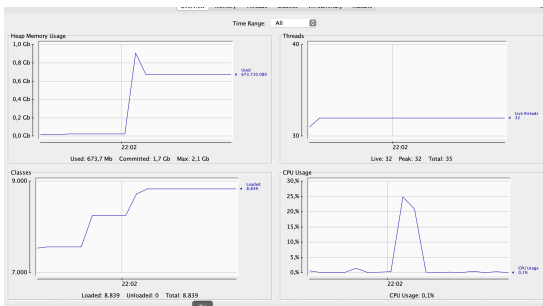
```
Generating rules from 18 frequent itemsets...
Generated 4 rules.
[TIME] Rule generation (Eclat): 4 ms
[TIME] Graph construction (nodes/edges): 1 ms
[TIME] GraphDt population: 0 ms
Applying PageRank...
[TIME] PageRank: 11 ms
[TIME] TOTAL execution time: 6664 ms
```

(c) Metrics for Eclat from Java Library

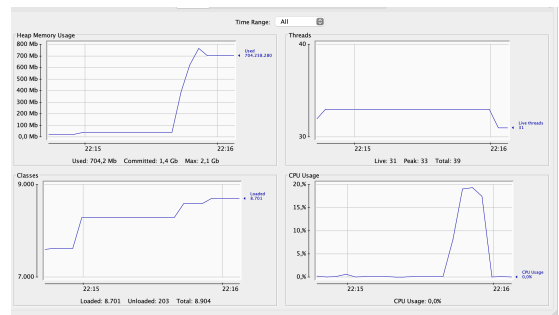
```
Apriori execution took: 74 ms
DONE FOR: 28 FILES
--- TIMING REPORT ---
Total Request: 13005 ms
-----
```

(d) Metrics for Apriori from Java Library

Following the same logical steps as we did in 9.2.1 the results that we get are:



(a) Eclat (new)



(b) Apriori (Legacy)

Further scaling of the dataset exposes the distinct operational signatures of the Eclat and Apriori algorithms, **highlighting a trade-off between transient resource intensity and final execution efficiency**. Eclat demonstrates a highly concentrated processing model, *characterized by a sharp, intense computational burst (25% CPU peak) and a rapid transient spike in heap memory allocation*. However, this aggressive, depth-first execution quickly resolves to a highly efficient resting memory footprint of 673.7 MB with **absolute thread stability**.

Conversely, Apriori exhibits a more prolonged, distributed computational load (19% CPU peak) and a gradual memory climb. While Apriori avoids Eclat's extreme transient memory spike, **it ultimately concludes with a higher resting memory state (704.2 MB)** and minor thread volatility. These profiles confirm that while Eclat demands higher immediate resource headroom to execute rapidly, it ultimately outperforms Apriori in post-execution memory efficiency and system stability.

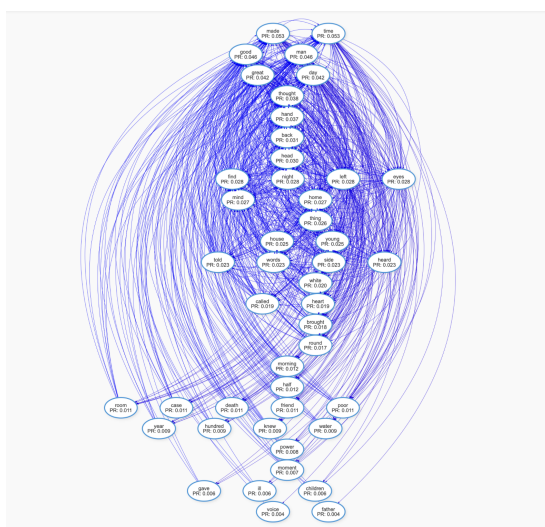
Dataset	Algorithm	Supp.	Conf.	Algo Time	Total Time	Memory Status
MIMIC-III (100MB)	Legacy (Apriori)	20%	20%	74ms	13005ms	704.2Mb - 19%
	Current (Eclat)	20%	20%	4ms	6664ms	673.7Mb - 25%

9.2.4 Gutenberg Books

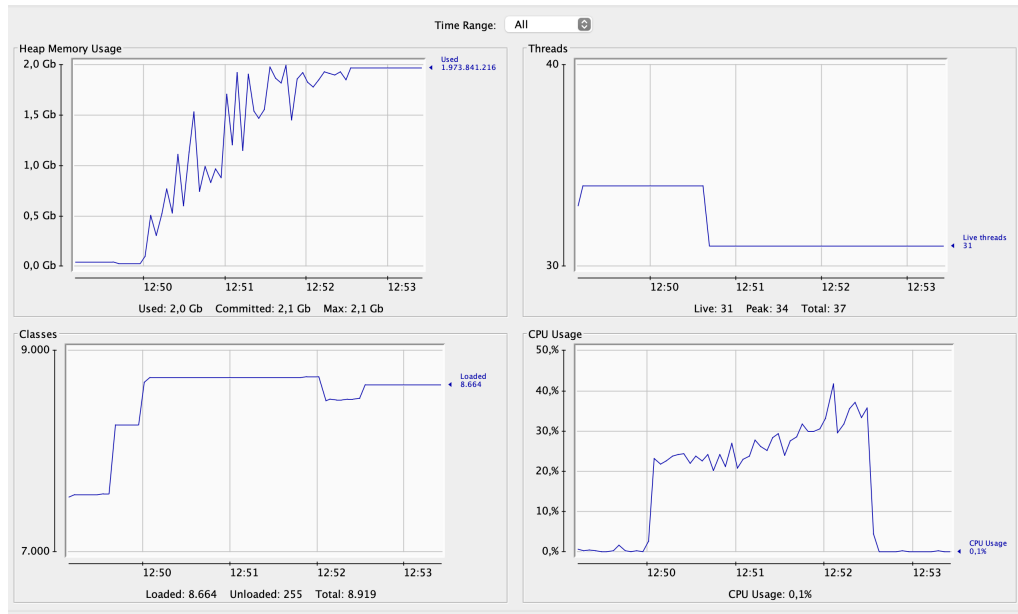
Unexpectedly for this Dataset i was not able to get results from the legacy app of *Efthimios Mitkousis*. Excessive tasting was made with different *Confidence*, *Support* and *Phrase Length* parameters. The only result we observed was **JAVA HEAP ERROR**:

```
2026-04-24T12:30:57.368+03:00 ERROR 22922 --- [nio-8080-exec-6] o.a.c.c.@@.[].[dispatcherServlet] : Servlet.service() for servlet [dispatcherServlet] in
context with path [] threw exception [Handler dispatch failed: java.lang.OutOfMemoryError: Java heap space] with root cause
java.lang.OutOfMemoryError: Create breakpoint : Java heap space
at java.base/java.lang.StringUTF16.newString(StringUTF16.java:1172) ~[na:na]
at java.base/java.lang.String.substring(String.java:2840) ~[na:na]
at java.base/java.lang.String.subSequence(String.java:2872) ~[na:na]
at java.base/java.util.regex.Pattern.split(Pattern.java:1412) ~[na:na]
at java.base/java.util.regex.Pattern.split(Pattern.java:1385) ~[na:na]
at java.base/java.lang.String.split(String.java:3359) ~[na:na]
at java.base/java.lang.String.split(String.java:3447) ~[na:na]
at org.example.View.VisualRestController.enterValues(VisualRestController.java:195) ~[classes:/na]
at java.base/java.lang.invoke.LambdaForm$MH/0x000000a08115c400.invokeVirtual(LambdaForm$MH) ~[na:na]
at java.base/java.lang.invoke.LambdaForm$MH/0x000000a08112d000.invoke(LambdaForm$MH) ~[na:na]
at java.base/java.lang.invoke.Invokers$Holder.invokeExact_MT(Invokers$Holder) ~[na:na] <13 internal lines>
at jakarta.servlet.http.HttpServlet.service(HttpServlet.java:590) ~[tomcat-embed-core-10.1.36.jar:6.0] <1 internal line>
at jakarta.servlet.http.HttpServlet.service(HttpServlet.java:650) ~[tomcat-embed-core-10.1.36.jar:6.0] <5 internal lines>
```

Furthermore, the current **Word Taxonomy Explorer** returned the following Taxonomy and results:



```
Generating rules from 488 frequent itemsets...
Generated 880 rules.
[TIME] Rule generation (Eclat): 82 ms
[TIME] Graph construction (nodes/edges): 17 ms
[TIME] GraphDt population: 1 ms
Applying PageRank...
[TIME] PageRank: 20 ms
[TIME] TOTAL execution time: 148555 ms
```



Evaluating the algorithms on a 500MB dataset highlights their respective behaviors **when subjected to strict hardware memory limitations (a 2.1 GB maximum heap)**. The Apriori algorithm’s breadth-first candidate generation proved structurally incompatible with these constraints. The combinatorial explosion of holding candidate sets in memory resulted in an out-of-memory failure.

Conversely, the Eclat algorithm completed the execution, though **it pushed the hardware to its absolute threshold**. Eclat’s depth-first approach allowed the JVM to just barely contain the memory footprint (*peaking at 2.0 GB*) through aggressive, continuous Garbage Collection and class unloading. However, **this survival came at the cost of severe resource saturation, characterized by heavy CPU thrashing (peaking over 40%)** and high execution volatility. Ultimately, under strict hardware constraints, Eclat demonstrates better survivability by trading sustained high CPU and GC overhead to prevent the fatal memory overloads inherent to Apriori’s design.

Dataset	Algorithm	Supp.	Conf.	Algo Time	Total Time	Memory Status
Gutenberg Books (500MB)	Legacy (Apriori)	NaN%	NaN%	NaN	NaN	NaN
	Current (Eclat)	80%	80%	82ms	2.48min	2.0GB - 40%

9.2.5 PDF Corpus

Following the last overwhelming result on the previous smaller dataset, the legacy app was not able to create a **Taxonomy** on our last and **biggest Dataset**.

```

2026-04-24T13:38:53.347+03:00 ERROR 23959 --- [nio-8080-exec-2] o.a.c.c.f.[...].dispatcherServlet : Servlet.service() for servlet [dispatcherServlet] in
context with path [] threw exception [Handler dispatch failed: java.lang.OutOfMemoryError: Java heap space] with root cause
java.lang.OutOfMemoryError: Create breakpoint : Java heap space
    at java.base/java.util.regex.Pattern.compile(Pattern.java:1934) ~[na:na]
    at java.base/java.util.regex.Pattern.<init>(Pattern.java:1574) ~[na:na]
    at java.base/java.util.regex.Pattern.compile(Pattern.java:1101) ~[na:na]
    at java.base/java.lang.String.replaceAll(String.java:3849) ~[na:na]
    at org.example.Taxonomy.Index.tokenize(Index.java:25) ~[classes/:na]
    at org.example.Taxonomy.Index.addCollection(Index.java:117) ~[classes/:na]
    at org.example.View.VisualRestController.EnterValues(VisualRestController.java:197) ~[classes/:na]
    at java.base/java.lang.invoke.LambdaForm$DMH/6x080808c88115c408.invokeVirtual(LambdaForm$DMH) ~[na:na]
    at java.base/java.lang.invoke.LambdaForm$MH/6x080808c88112d008.invoke(LambdaForm$MH) ~[na:na]
    at java.base/java.lang.invoke.Invokers$Holder.invokeExact_MT(Invokers$Holder) ~[na:na] <3 internal lines>
    at jakarta.servlet.http.HttpServlet.service(HttpServlet.java:590) ~[tomcat-embed-core-10.1.36.jar:6.0] <1 internal lines>
    at jakarta.servlet.http.HttpServlet.service(HttpServlet.java:658) ~[tomcat-embed-core-10.1.36.jar:6.0] <6 internal lines>

```

An identical observation was made while trying to run our last **Dataset**, on the new App using the Eclat Algorithm.

```
Exception in thread "http-nio-8080-Poller" java.lang.OutOfMemoryError: Create breakpoint : Java heap space
    at java.base/java.util.Collections$UnmodifiableCollection.iterator(Collections.java:1073) <2 internal lines>
    at java.base/java.lang.Thread.run(Thread.java:1595) <1 internal lines>
2026-04-24T14:05:50.718+03:00 ERROR 24700 --- [nio-8080-exec-1] o.a.c.c.C.[.[/].[dispatcherServlet] : Servlet.service() for servlet [dispatcherServlet] in
context with path [] threw exception [Handler dispatch failed: java.lang.OutOfMemoryError: Java heap space] with root cause

java.lang.OutOfMemoryError: Create breakpoint : Java heap space
    at java.base/java.util.HashMap.newNode(HashMap.java:1909) ~[na:na]
    at java.base/java.util.HashMap.putVal(HashMap.java:637) ~[na:na]
    at java.base/java.util.HashMap.put(HashMap.java:618) ~[na:na]
    at java.base/java.util.HashSet.add(HashSet.java:229) ~[na:na]
    at org.example.Taxonomy.Index.addCollection(Index.java:107) ~[classes:na]
    at org.example.View.VisualRestController.EnterValues(VisualRestController.java:326) ~[classes:na]
    at java.base/java.lang.invoke.LambdaForm$DMH/0x0000000000000000.invokeVirtual(LambdaForm$DMH) ~[na:na]
    at java.base/java.lang.invoke.LambdaForm$DMH/0x0000000000000000.invoke(LambdaForm$DMH) ~[na:na]
    at java.base/java.lang.invoke.Invokers$Holder.invokeExact_MT(Invokers$Holder) ~[na:na] <13 internal lines>
    at jakarta.servlet.http.HttpServlet.service(HttpServlet.java:590) ~[tomcat-embed-core-10.1.36.jar:6.0] <1 internal lines>
    at jakarta.servlet.http.HttpServlet.service(HttpServlet.java:658) ~[tomcat-embed-core-10.1.36.jar:6.0] <7 internal lines>
```

The conclusion is that, **both algorithmic approaches fail while trying to processes big sets of data, on a lower hardware PC**

Dataset	Algorithm	Supp.	Conf.	Algo Time	Total Time	Memory Status
PDF Corpus (2.4GB)	Legacy (Apriori)	NaN%	NaN%	NaN	NaN	NaN
	Current (Eclat)	NaN%	NaN%	NaN	NaN	NaN

9.2.6 Full Experiment Results

Dataset	Algorithm	Supp.	Conf.	Algo Time	Total Time	Memory Status
SMcollection (64KB)	Apriori	40%	40%	5ms	157ms	220,7Mb - 3.2%
	Eclat	40%	40%	3ms	145ms	203,9Mb - 2.6%
20 News Groups (40MB)	Apriori	10%	10%	486ms	15580ms	655.8Mb - 16%
	Eclat	10%	10%	16ms	10815ms	869.7Mb - 16%
MIMIC-III (100MB)	Apriori	20%	20%	74ms	13005ms	704.2Mb - 19%
	Eclat	20%	20%	4ms	6664ms	673.7Mb - 25%
Guteberg Books (500MB)	Apriori	NaN%	NaN%	NaN	NaN	NaN
	Eclat	80%	80%	82ms	2.48min	2.0GB - 40%
PDF Corpus (2.4GB)	Apriori	NaN%	NaN%	NaN	NaN	NaN
	Eclat	NaN%	NaN%	NaN	NaN	NaN

- Algorithmic Speed and Total Execution Time:**

- Massive Eclat Advantage:** The most striking difference is in *"Algo Time"*. As datasets grow beyond the trivial 64KB mark, Eclat becomes exponentially faster. For the 40MB dataset, Eclat's algorithmic processing took 16ms compared to Apriori's 486ms. For the 100MB dataset, Eclat took just 4ms compared to Apriori's 74ms.
- Total Time Savings:** This algorithmic efficiency translates directly to *"Total Time"*. Across all successful comparative runs, Eclat finished the end-to-end process significantly faster (*e.g., ≈ 6.6 seconds vs ≈ 13.0 seconds on the 100MB MIMIC-III dataset*).

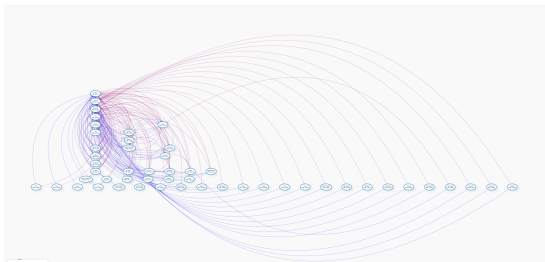
- Memory and Computational Trade-offs**

1. **The CPU Tax:** Eclat's speed often comes at the cost of higher CPU utilization. On the *MIMIC-III dataset*, Eclat required 25% CPU compared to Apriori's 19%, reflecting the intense computational burst required for its depth-first intersections.

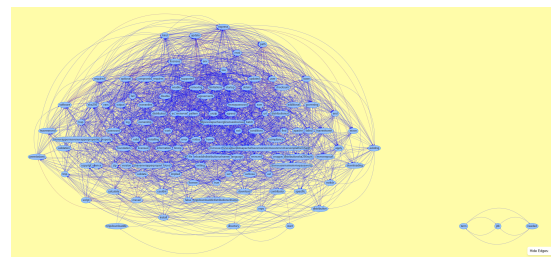
9.3 System Experiments on Windows

To conclude the evaluation, the datasets were processed on *an alternative hardware configuration* to benchmark the application's scalability and portability. The objective of this comparative study was twofold: first, to measure the reduction in processing latency achieved by the new implementation, and second, to verify if the updated algorithms produced more robust and accurate taxonomies compared to the baseline environment.

9.3.1 SMCollection



(a) Current Graph



(b) Legacy Graph

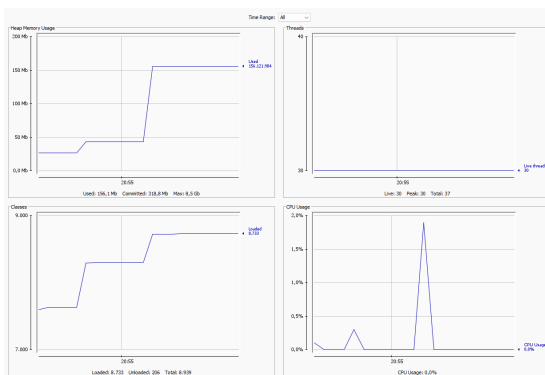
```
Generating rules from 184 frequent itemsets...
Generated 224 rules.
[TIME] Rule generation (Eclat): 19 ms
[TIME] Graph construction (nodes/edges): 9 ms
[TIME] GraphDt population: 2 ms
Applying PageRank...
[TIME] PageRank: 14 ms
[TIME] TOTAL execution time: 143 ms
```

(c) Metrics for Eclat from Java Library

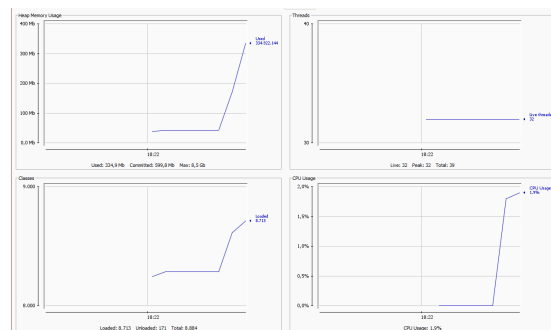
```
TOO MANY RULES
[TIME] APRIORI ALGORITHM TIME 1227 MS
DONE FOR: 6 FILES
---TOTAL TIMING REPORT---
[TIME] TOTAL REQUEST TIME 2148 MS
```

(d) Metrics for Apriori from Java Library

Following the same logical steps as we did in 9.2.1 the results that we get are:



(a) Eclat (new)



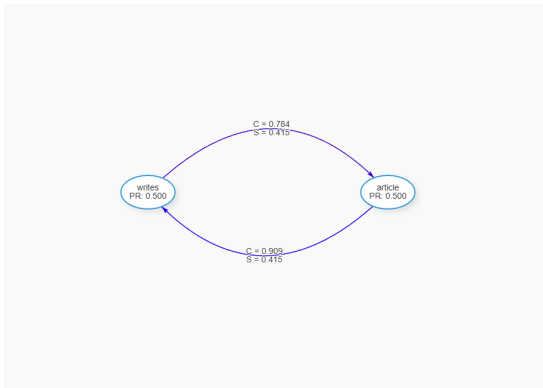
(b) Apriori (Legacy)

Migrating the experimental setup to a higher-capacity Windows environment isolated the algorithms from strict hardware bottlenecks, revealing their baseline resource efficiencies. Under these unconstrained conditions, the new system architecture **Eclat demonstrated a vastly superior memory utilization profile**. It successfully executed the workload requiring a peak heap footprint of only *156.1 MB*, consuming less than half the memory of the legacy Apriori system *334.9 MB*.

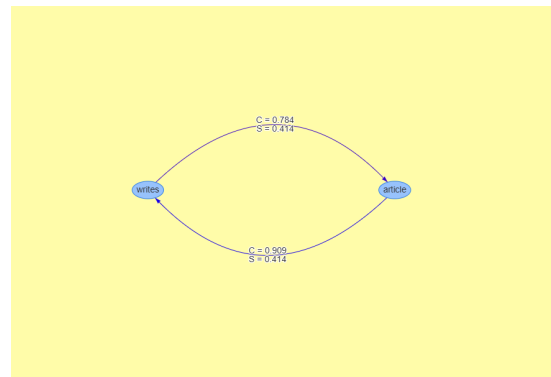
Furthermore, while the increased computational power of the test environment **reduced CPU strain to nominal levels ($\tilde{1.9\%}$) for both systems**, the new architecture maintained a perfectly stable, slightly smaller thread pool (*30 live threads* vs. *32*). **These results confirm that the architectural enhancements of the new system extend beyond mere survival under low-memory constraints. It fundamentally operates with a vastly lighter, more efficient baseline resource footprint, on smaller and lighter datasets such as *SMCollection*.**

Dataset	Algorithm	Supp.	Conf.	Algo Time	Total Time	Memory Status
SMcollection (64KB)	Apriori	30%	30%	1227ms	2148ms	334,9Mb - 1.9%
	Eclat	30%	30%	19ms	143ms	156,1Mb - 1.9%

9.3.2 20 News Groups Collection



(a) Current Graph



(b) Legacy Graph

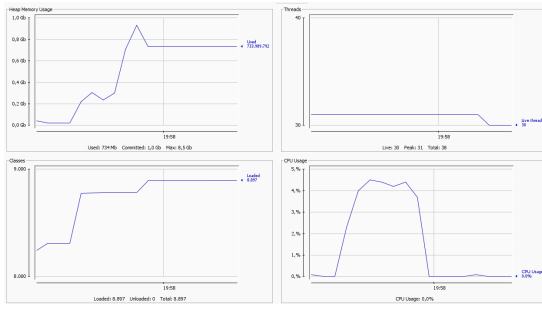
```
[TIME] Rule generation (Eclat): 14 ms
[TIME] Graph construction (nodes/edges): 1 ms
[TIME] GraphDt population: 0 ms
Applying PageRank...
[TIME] PageRank: 5 ms
[TIME] TOTAL execution time: 23972 ms
```

(c) Metrics for Eclat from Java Library

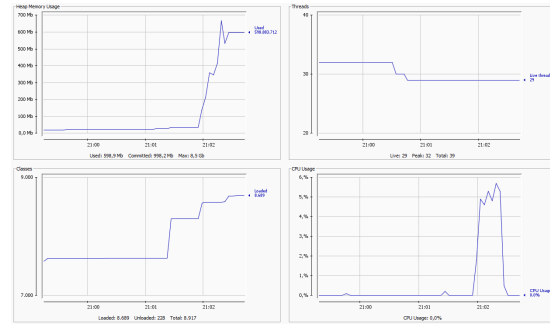
```
[TIME] APRIORI ALGORITHM TIME 105 MS
DONE FOR: 19997 FILES
---TOTAL TIMING REPORT---
[TIME] TOTAL REQUEST TIME 25415 MS
```

(d) Metrics for Apriori from Java Library

Following the same logical steps as we did in 9.2.1 the results that we get are:



(a) Eclat (new)



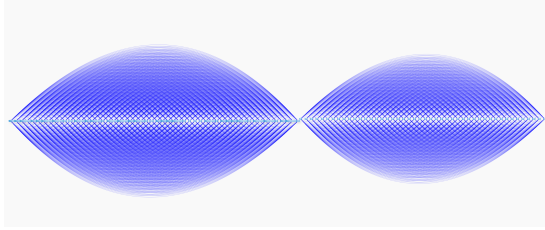
(b) Apriori (Legacy)

Testing an up-scaled workload within an unconstrained hardware environment **exposes a fundamental architectural trade-off between the new and legacy systems**. Without the threat of out-of-memory errors, the legacy system (*Apriori*) achieves a lower final memory footprint (598.9 MB). However, this comes at the cost of execution volatility, evidenced by a jagged CPU profile and active class unloading indicative of continuous Garbage Collection churn.

In contrast, the new system architecture (*Eclat*) leverages the available hardware to prioritize execution stability. It trades a higher resting memory footprint (734.0 MB) to accommodate its vertical data structures, which enables a remarkably smooth, uninterrupted CPU processing phase peaking at 4.5%. **These findings demonstrate that in high-capacity environments, the new system effectively utilizes additional memory overhead to bypass the volatile, stop-the-world resource management cycles that inherently throttle the legacy algorithm.**

Dataset	Algorithm	Supp.	Conf.	Algo Time	Total Time	Memory Status
20 News Groups (40MB)	Legacy (Apriori)	20%	20%	105ms	25415ms	655.8Mb - 16%
	Current (Eclat)	20%	20%	14ms	23972ms	734Mb - 4.5%

9.3.3 MIMIC-III



(a) Current Graph



(b) Legacy Graph

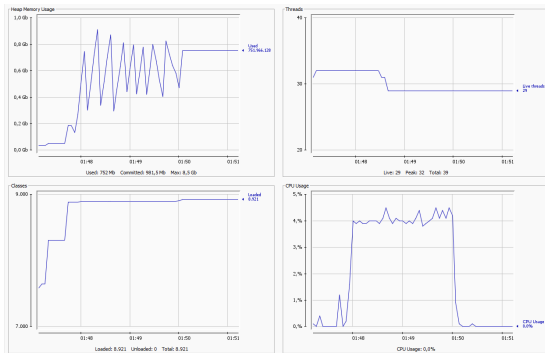
```
Generating rules from 158565 frequent itemsets...
Generated 315902 rules.
TOO MANY RULES
[TIME] Rule generation (Eclat): 808 ms
[TIME] Graph construction (nodes/edges): 122074 ms
[TIME] GraphDt population: 330 ms
Applying PageRank...
[TIME] PageRank: 1945 ms
[TIME] TOTAL execution time: 125179 ms
```

(c) Metrics for Eclat from Java Library

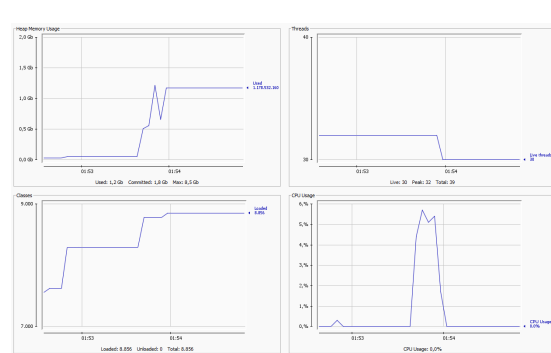
```
[TIME] APRIORI ALGORITHM TIME 3477 MS
DONE FOR: 28 FILES
---TOTAL TIMING REPORT---
[TIME] TOTAL REQUEST TIME 16792 MS
```

(d) Metrics for Apriori from Java Library

Following the same logical steps as we did in 9.2.1 the results that we get are:



(a) Eclat (new)



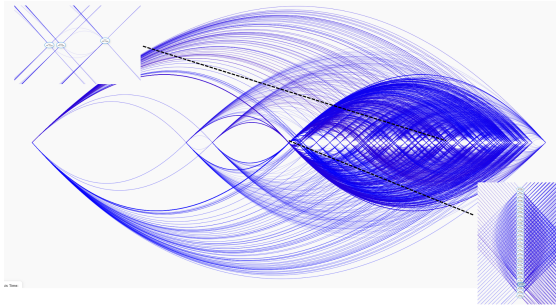
(b) Apriori (Legacy)

The legacy Apriori system suffered a complete silent failure. While its execution profile shows it consumed significant resources—peaking at a 5.5% CPU load and settling at an inflated 1.17 GB memory footprint, it ultimately failed to generate the target graph entities. **This indicates either that Apriori’s breadth-first candidate generation fundamentally breaks down on this data complexity. Or, the structure of the dataset itself might cause the App to silent fail.**

The Eclat based architecture proved functionally robust. Its execution required sustained computational effort (*a prolonged 4.5% CPU plateau*) and intensive memory management, evidenced by a severe sawtooth Garbage Collection pattern handling transient data structures. However, this aggressive processing was successful, yielding graph entities and ultimately resolving to a much lower resting memory footprint of 752 MB. **This confirms that the new system is not merely more efficient, but strictly necessary for achieving functional viability on complex, real-world taxonomic datasets.**

Dataset	Algorithm	Supp.	Conf.	Algo Time	Total Time	Memory Status
MIMIC-III (100MB)	Legacy (Apriori)	50%	50%	3477ms	16792ms	1.17GB - 5.5%
	Current (Eclat)	50%	50%	808ms	125179ms	752Mb - 4.5%

9.3.4 Gutenberg Books



(a) Current Graph



(b) Legacy Graph

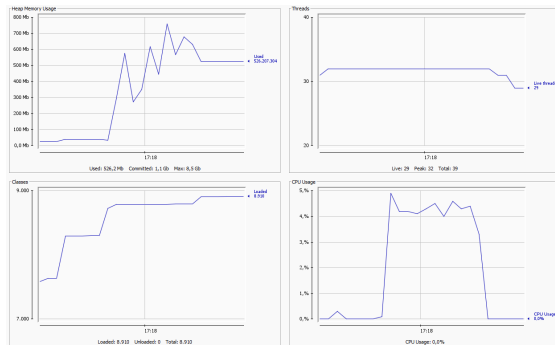
```
Generated 13862 rules.
TOO MANY RULES
[TIME] Rule generation (Eclat): 174 ms
[TIME] Graph construction (nodes/edges): 989 ms
[TIME] GraphDt population: 25 ms
Applying PageRank...
[TIME] PageRank: 113 ms
[TIME] TOTAL execution time: 42185 ms
```

(c) Metrics for Eclat from Java Library

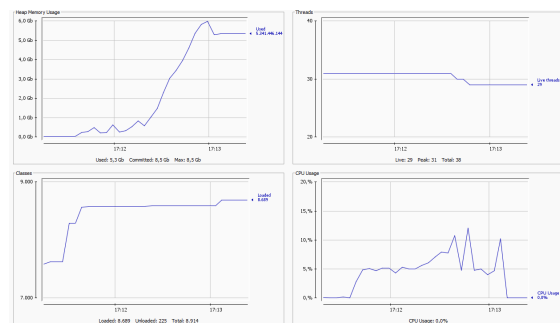
```
TOO MANY RULES
[TIME] APRIORI ALGORITHM TIME 45444 MS
DONE FOR: 146 FILES
---TOTAL TIMING REPORT---
[TIME] TOTAL REQUEST TIME 87370 MS
```

(d) Metrics for Apriori from Java Library

Following the same logical steps as we did in 9.2.1 the results that we get are:



(a) Eclat (new)



(b) Apriori (Legacy)

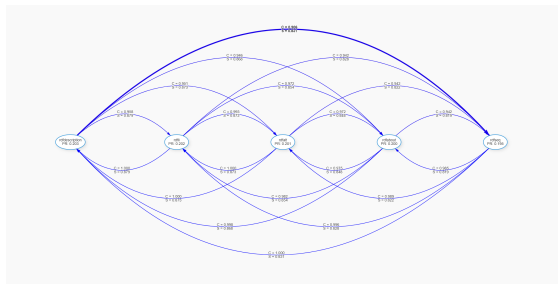
Following the second to last test on the **Windows PC**, concluding from both of the above usage information of the systems. The legacy system exhibited a **massive expansion in memory consumption**. Its heap profile shows a massive, continuous climb, peaking near **6.0 GB** before resting at **5.3 GB**. This demonstrates the severe combinatorial explosion inherent in Apriori's **breadth-first candidate generation** when processing a real-world free text set of data. Studying the current system now, it handled the identical workload with a highly contained sawtooth memory profile, **resting at a mere 526.2 MB**. The new system achieved the goal using approximately **1/10th** of the memory required by the legacy system.

From the **execution duration and CPU volatility** side - *our current system* - displayed a concise, highly stable execution phase. Its CPU usage formed a clean, sustained plateau between 4% and 5% before immediately resolving, indicating focused, uninterrupted algorithmic computation. On the other side - *the legacy system* - suffered a severely prolonged and volatile execution phase. Its CPU profile is highly jagged, ramping up to peaks of 12% over a much longer duration. This volatility, paired with the massive memory footprint, strongly indicates that the JVM was spending massive amounts of computational power simply traversing and garbage-collecting gigabytes of transient candidate itemsets.

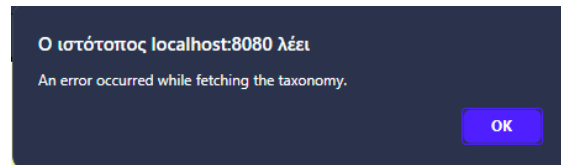
Dataset	Algorithm	Supp.	Conf.	Algo Time	Total Time	Memory Status
Gutenberg Books (500MB)	Legacy (Apriori)	60%	60%	45444ms	87370ms	5.3GB - 12%
	Current (Eclat)	60%	60%	174ms	42185ms	526.2MB - 5%

9.3.5 PDF Corpus

While trying to run the same Corpus on both programs, the legacy app was unable to provide a **Taxonomy** with the base VM settings provided by the IDE that we used - *IntelliJ IDEA 2023.3.4* -. Expanding on that, the baseline **Heap Space** provided by the App was **17.2GBs**, since that was not enough after finding the **highest Heap Space value** that i could provide the VM - *according to our current hardware setup explained in 9.1.1* - i updated the **Run Configurations** with the option: `-Xmx20g` thus having the max **Heap Space of 21.5GBs**. Even with those settings the results of the first test were:



(a) Current Graph



(b) Legacy Graph

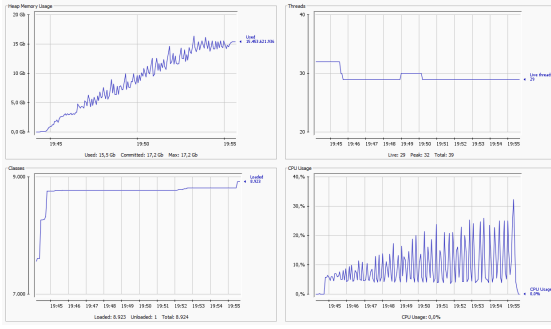
```
Processed 651 transactions (DISJUNCT level)
[TIME] File reading + segmentation: 621702 ms
[TIME] FP-UP calculation: 22923 ms
Starting Eclat algorithm...
Absolute support threshold: 520.8000000000001
Maximum phrase length is: 1
Found 9 frequent itemsets
Eclat mining finished in 1 ms.
2020-05-08T19:35:19.244+03:00 [INFO] 10992 --- [nio-8080-exec-2] s.stanford.nlp.pipeline.StanfordCoreNLP : Adding annotator tokenize
2020-05-08T19:35:19.244+03:00 [INFO] 10992 --- [nio-8080-exec-2] s.stanford.nlp.pipeline.StanfordCoreNLP : Adding annotator pos
Generating rules from 17 frequent itemsets...
Generated 20 rules.
[TIME] Rule generation (Eclat): 22 ms
[TIME] Graph construction (nodes/edges): 5 ms
[TIME] Graphviz population: 0 ms
Applying PageRank...
[TIME] PageRank: 5 ms
[TIME] TOTAL execution time: 64676 ms
```

(c) Metrics for Eclat from Java Library

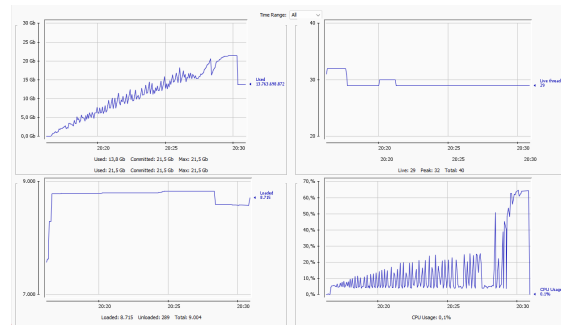
```
2020-05-08T19:35:19.244+03:00 [INFO] 10992 --- [nio-8080-exec-2] s.stanford.nlp.pipeline.StanfordCoreNLP : Adding annotator tokenize
2020-05-08T19:35:19.244+03:00 [INFO] 10992 --- [nio-8080-exec-2] s.stanford.nlp.pipeline.StanfordCoreNLP : Adding annotator pos
Generating rules from 17 frequent itemsets...
Generated 20 rules.
[TIME] Rule generation (Eclat): 22 ms
[TIME] Graph construction (nodes/edges): 5 ms
[TIME] Graphviz population: 0 ms
Applying PageRank...
[TIME] PageRank: 5 ms
[TIME] TOTAL execution time: 64676 ms
```

(d) Metrics for Apriori from Java Library

Following the same logical steps as we did in 9.2.1 the results that we get are:



(a) Eclat (new)

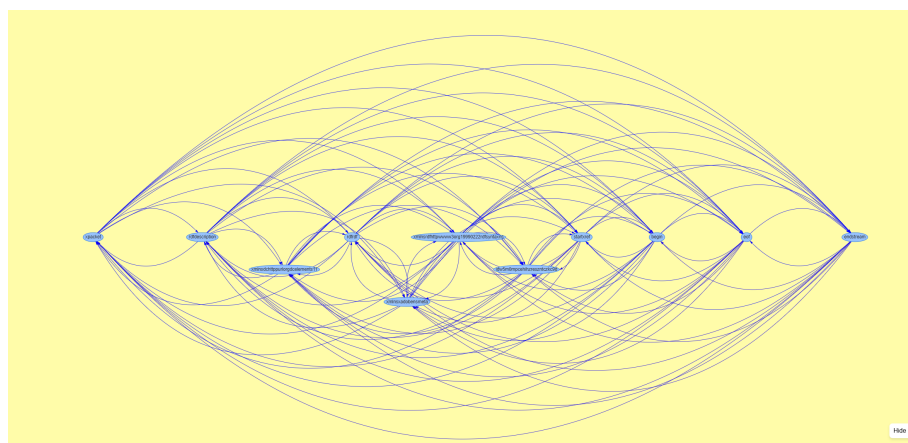


(b) Apriori (Legacy)

Subjecting the architectures to the massive PDF Corpus dataset firmly establishes the practical, operational superiority of the Eclat algorithm. The legacy Apriori system proved incapable of functioning within the baseline *17.2 GB JVM heap limit*. Even after manually overriding the run configurations (`-Xmx20g`) to allocate an extreme *21.5 GB heap boundary*, Apriori exhibited critical system distress. **Its execution profile collapsed into severe Garbage Collection (GC) thrashing—evidenced by CPU spikes exceeding 60% and emergency class unloading—as the hardware struggled to manage the algorithm’s combinatorial memory explosion.**

Conversely, the new Eclat architecture successfully processed the identical Corpus without requiring any manual hardware overrides. It operated entirely within the baseline *17.2 GB limit*, peaking at *15.4 GB of utilized heap*. Its execution remained computationally structured and rhythmic (*peaking at 32% CPU*) without succumbing to the fatal GC thrashing that paralyzed the legacy system. **This definitively proves that the Eclat implementation is not just computationally faster, but is the only architecturally viable solution for processing extreme-scale taxonomic datasets without causing total resource exhaustion.**

Despite this unsuccessful last attempt by the legacy app, *we decided to re-run the same Dataset but with higher **Support & Confidence** values* to make the creation of the Taxonomy "easier".

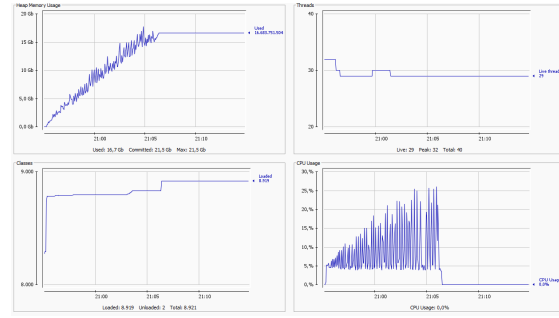


```

[TIME] APRIORI ALGORITHM TIME 6746 MS
DONE FOR: 651 FILES
---TOTAL TIMING REPORT---
[TIME] TOTAL REQUEST TIME 668095 MS

```

(a) Timing report on new Test



(b) Usage report on new Test

Final testing on the massive PDF Corpus reveals a critical limitation of the legacy Apriori architecture: **the necessity for analytical compromise**. To achieve a successful execution and generate a taxonomy, the legacy system required both a manual expansion of the JVM heap limit (*to 21.5 GB*) and an **artificial reduction** in algorithmic difficulty via relaxed support and confidence thresholds. Even with this computationally 'easier' workload, Apriori consumed a peak heap footprint of *16.6 GB*.

Dataset	Algorithm	Supp.	Conf.	Algo Time	Total Time	Memory Status
PDF Corpus (2.4GB)	Legacy (Apriori)	90%	90%	6746ms	11.13min	16.7GB - 26%
	Current (Eclat)	80%	80%	22ms	10.74min	15.5GB - 33%

9.3.6 Full Experiment Results

Dataset	Algorithm	Supp.	Conf.	Algo Time	Total Time	Memory Status
SMcollection (64KB)	Apriori	30%	30%	1227ms	2148ms	445.9Mb - 1.9%
	Eclat	30%	30%	19ms	143ms	156.1Mb - 1.9%
20 News Groups (40MB)	Apriori	20%	20%	105ms	25415ms	655.8Mb - 16%
	Eclat	20%	20%	14ms	23972ms	734Mb - 4.5%
MIMIC-III (100MB)	Apriori	50%	50%	3477ms	16792ms	1.17GB - 5.5%
	Eclat	50%	50%	808ms	12179ms	752Mb - 4.5%
Guteberg Books (500MB)	Apriori	60%	60%	45444ms	87370ms	5.3GB - 12%
	Eclat	60%	60%	174ms	42185ms	526.2MB - 5%
PDF Corpus (2.4GB)	Apriori	90%	90%	6746ms	11.13min	16.7GB - 26%
	Eclat	80%	80%	22ms	10.74min	15.5GB - 33%

10 System Usage Instructions

In its core the operation of the **Word Taxonomy Explorer**, did not move further away from its original usage from Efthimios Mitkousis [20]. That said, this chapter serves as a comprehensive operational guide. It details the practical application of the system, outlining how end-users and researchers **can interact with the underlying data mining algorithms to extract, visualize, and interpret semantic relationships from unstructured text**.

Prerequisites

Prior to launching the application, the host machine must meet specific environmental prerequisites and compile the source code via the build management tool.

- **Java Development Kit (JDK):** The system requires JDK 17 or a later version to support modern language features and the Spring Boot framework.
- **Apache Maven:** Utilized for dependency management and project compilation.
- **Data Corpus:** A local directory containing the target unstructured text. The system natively supports .txt files and .pdf documents.

System Setup and Execution

To initialize the system, clone the repository and compile the executable using the following terminal commands:

```
1 # Clone the remote repository
2 git clone https://github.com/vaggelisMichelioudakis/Deriving-
  interactive-Visual-Association-Maps-From-Documents-Using-
  Association-Rule-Mining.git
3
4 # Navigate to the root directory
5 cd AssociationMaps
6
7 # Clean previous builds and compile the project
8 mvn clean install
```

Running the Program

Once the build is successful, the embedded Apache Tomcat server can be initiated using either the compiled Java Archive (JAR) or the Spring Boot Maven plugin:

```
1 java -jar target/WebApp1-1.0-SNAPSHOT.jar
2 # OR
3 mvn spring-boot:run
```

Upon initialization, the application binds to port 8080. The graphical interface can be accessed by navigating to <http://localhost:8080/visualize> in any modern web browser.

Web Application Configuration

The graphical interface utilizes a Split-Pane Dashboard design. To generate a taxonomy, the user must provide the initial constraints via the Control Sidebar:

1. **Folder Path:** Select the absolute local directory path containing the target corpus.
2. **Granularity Context:** Defines the transactional boundary for the mining algorithm. Users can select between *Document*, *Paragraph*, *Sentence*, or *Semantic Chunk* levels based on the desired strictness of co-occurrence.
3. **Support Threshold:** Establishes the minimum occurrence frequency required for a rule. This can be expressed as a relative percentage (*e.g.*, *0.1 for 10%*) or an absolute integer representing document count.
4. **Confidence Threshold:** Defines the conditional probability required for an edge. A value of 0.7 requires that the consequent term appears in 70% of the transactions containing the antecedent term.
5. **Phrase Length (Maximum Depth):** Specifies the maximum *n*-gram length for the extracted terms (*e.g.*, *a value of 2 allows for the discovery of bigrams like "machine learning"*).

Interactive Map Navigation

Following the mathematical processing, the taxonomy is rendered onto the primary canvas as a force-directed graph. The environment supports the following interactive evaluations:

- **Explore Terms (Nodes):** Clicking on any conceptual node opens the *Node Info* panel, detailing its calculated hierarchical level, Weighted PageRank score, and interconnected dependencies.
- **Inspect Rules (Edges):** Clicking on a connecting line between concepts opens the *Evidence Modal*. This displays the statistical strength (Confidence and Lift) alongside the extracted raw text *Snippets*, proving exactly where the terms co-occurred in the original documents.
- **Live State Updates:** Users can utilize the live view filters to dynamically clarify the graph without requiring a full system restart or corpus re-indexing.

Tips for Optimal Analysis

To maximize the efficacy of the Word Taxonomy Explorer, users should adhere to the following exploratory data analysis (EDA) practices:

- **Data Sanitization:** Ensure the target folder contains cleanly formatted text files to prevent optical character recognition (OCR) artifacts or encoding errors from skewing the tokenization phase.
- **Iterative Thresholding:** It is recommended to initiate the mining process with highly permissive Support and Confidence thresholds to map the broad structure of the corpus. The user can then progressively tighten these thresholds using the live filters to isolate the most robust semantic rules.
- **Granularity Experimentation:** If the resulting network is overly sparse, adjusting the Granularity from *Sentence* to *Paragraph* can capture broader thematic relationships that rigid punctuation boundaries might obscure.

11 Conclusion

11.1 Summary

Following the completion of this Thesis, it presented a comprehensive re-engineering of the **Word Taxonomy Explorer**, successfully transforming it from a static, computationally restricted script into *a high-performance, scalable visual analytics dashboard*. The core objective of this work was realized through a complete architectural overhaul of both the backend computational engine and the frontend presentation layer.

The most significant theoretical and practical upgrade was **the migration from the Apriori algorithm to the memory-efficient Eclat algorithm**. By abandoning Apriori’s horizontal data layout and exhaustive breadth-first candidate generation, the new system leverages vertical data formats and depth-first transaction intersections. **This eliminates the massive I/O bottlenecks and "candidate explosion" problems that previously crippled the system’s ability to process long-form text.**

Beyond the core mining algorithm, the system’s analytical rigor was enhanced in two major ways:

- **The introduction of the Granularity Enhancement module** — *specifically the use of Jaccard similarity for Semantic Chunking* — ensures that transactions are defined by genuine conceptual continuity rather than arbitrary punctuation.
- The taxonomy’s hierarchical integrity was upgraded by replacing a susceptible local degree centrality heuristic with a robust, **Global Weighted PageRank** algorithm that accurately distributes conceptual "*authority*" based on rule confidence.

To summarize the sweeping nature of these upgrades, the following Table outlines the fundamental architectural differences between the legacy and current systems.

System Component	Legacy Approach	Current Approach (Updated System)
Mining Algorithm	Apriori (Breadth-First Search)	Eclat (Depth-First Search)
Data Layout	Horizontal (Transaction → Items)	Vertical (Item → Transaction IDs)
Context Definition	Static File-Level split by whitespace	Dynamic Granularity (Semantic Chunking via Jaccard)
Node Importance	Local Degree Centrality (In/Out-Degree)	Global Weighted PageRank
User Interface	Monolithic linear document	Event-Driven Split-Pane Dashboard
Explainability	Abstract nodes only	Contextual Evidence Modals (Text Snippets)

11.2 Deep Dive: Experimental and Empirical Comparisons

To rigorously validate the architectural shifts, the system underwent comparative testing against its predecessor **across multiple datasets** ranging from a *64KB synthetic text file* to a *2.4GB PDF corpus*. To ensure a comprehensive evaluation of both survival under constraints and baseline algorithmic efficiency, tests were distinctly categorized by their hardware environment: *a constrained mobile setup (macOS M1)* and *a high-capacity workstation (Windows i9) 9.1.1*.

11.2.1 Performance Under Hardware Constraints (macOS M1)

The macOS testing environment (*Apple M1, 8GB Unified RAM*) served as a strict bottleneck to evaluate how both systems handle resource limitations.

- **Algorithmic Speed:** Eclat **demonstrated** a massive, exponential **speed advantage** as dataset dimensionality increased. On the *40MB 20 News Groups corpus*, Eclat's algorithmic processing was an astounding **30x faster** than Apriori (*16ms compared to 486ms*). This dominance continued on the *100MB MIMIC-III dataset*, where Eclat's rule generation was over **18x faster** (*4ms compared to 74ms*). This raw computational efficiency translated directly to total execution time. The current system finished the *MIMIC-III* pipeline nearly **2x faster** end-to-end (*6.6s versus 13.0s*).
- **The Candidate Explosion and Survival:** The most critical failing of the legacy system was exposed during the *500MB Gutenberg corpus* test. Apriori's breadth-first approach requires holding massive amounts of generated candidate itemsets in the JVM heap, **resulting in a fatal `OutOfMemoryError` and complete system crash**. Under these same strict conditions, Eclat survived. By leveraging its depth-first approach, Eclat aggressively utilized class unloading and intense CPU/Garbage Collection thrashing to contain the memory footprint (*peaking at 2.0 GB*), successfully mapping the taxonomy in **2.48 minutes** and preventing a crash.

Dataset Size	System / Algorithm	Algorithmic Time	Total Execution Time
64KB (SMCollection)	Legacy (Apriori)	5ms	157ms
	Current (Eclat)	3ms	145ms
40MB (20 News Groups)	Legacy (Apriori)	486ms	15,580ms
	Current (Eclat)	16ms	10,815ms
100MB (MIMIC-III)	Legacy (Apriori)	74ms	13,005ms
	Current (Eclat)	4ms	6,664ms
500MB (Gutenberg)	Legacy (Apriori)	<i>Failed (OOM)</i>	<i>Failed (OOM)</i>
	Current (Eclat)	82ms	148,555ms

11.2.2 Baseline Resource Efficiency (Windows i9)

Migrating the experimental setup to a high-capacity environment (*Intel Core i9, 32GB RAM*) removed the threat of out-of-memory errors, revealing the true, unconstrained resource footprints of both algorithms. **Arguably, the results done on this environment do present a high degree of interest.**

Observation to pure Algorithmic Acceleration can be made. When completely unconstrained by hardware limits, the speed differential between the two algorithms is staggering. Even on the smallest 64KB dataset (*SMCollection*), the new Eclat algorithm completed rule generation in **19ms** compared to Apriori's **1,227ms** — making the new backend roughly **64x** faster right out of the gate.

However, the true scale of this improvement is realized **as data dimensionality expands**. On the *500MB Gutenberg Books* corpus, the legacy Apriori system agonizingly churned through its breadth-first candidate generation for **45,444ms**. The updated Eclat algorithm handled the exact same dataset's rule generation in just **174ms**. This represents an incredible **261x acceleration** in core algorithmic processing at scale. This raw efficiency cascades into the total system execution, where the current system mapped the entire *500MB Gutenberg* taxonomy more than **2x faster** end-to-end (**42.1 seconds** versus **87.3 seconds**).

From a Memory Efficiency stand. The Windows experiments definitively proved that the legacy system's memory bloat was not a symptom of limited hardware, but a fatal architectural flaw. When given unlimited headroom to process the *500MB Gutenberg* corpus, Apriori's heap profile exhibited **a massive, continuous climb**, peaking and resting at an inflated **5.3 GB**. Furthermore, this execution was highly volatile, characterized by jagged **CPU spikes up to 12%**, indicating that the JVM was expending massive computational power simply *garbage-collecting gigabytes of transient, invalid candidate itemsets*.

In stark contrast, the current system handled the identical *500MB workload* with a clean, sustained CPU plateau (*peaking at only 5%*) and a highly contained sawtooth memory profile that rested at a mere **526.2 MB**. By transitioning to a vertical data layout and depth-first search, the new system achieved the goal using *exactly 1/10th of the memory required by the legacy system*.

Beyond speed and memory, **System Reliability is an important measurement**. The unconstrained tests highlighted a critical disparity in fundamental reliability. During the *100MB MIMIC-III* dataset evaluation, the legacy Apriori system consumed significant computational resources — climbing to a **1.17 GB** memory footprint—yet suffered a "*complete silent failure*" where it ultimately **failed to generate any target graph entities**. The Eclat-based architecture, however, proved functionally robust. It successfully resolved the complex medical dataset while settling at a vastly lower resting memory footprint of **752 MB**. *This confirms that the new system is not merely an optimization, but a strict requirement for achieving functional viability on complex, real-world data.*

The final benchmark for both systems came in the form of the PDF Corpus (2.4GB). The legacy system proved fundamentally incapable of functioning within the workstation's baseline *17.2 GB JVM heap limit*. **To force a successful execution and generate a taxonomy, the legacy system required two drastic interventions.** First, it required a manual hardware override of the

run configurations (`-Xmx20g`) to allocate an extreme *21.5 GB heap boundary*. Second, it required an artificial reduction in algorithmic difficulty via relaxed **Support** and **Confidence** thresholds. Even with this computationally "*easier*" workload and artificially inflated memory ceiling, Apriori exhibited critical system distress. Its execution profile collapsed into severe Garbage Collection (GC) thrashing—evidenced by emergency class unloading and CPU spikes exceeding **30%** — as the hardware struggled to manage the algorithm’s combinatorial memory explosion, ultimately peaking at *16.6 GB* of utilized heap.

Conversely, the new Eclat architecture successfully processed the identical, uncompromised PDF Corpus without requiring any manual hardware overrides or parameter relaxation. **As evidenced by the JVM profiling metrics, it operated safely entirely within the baseline 17.2 GB limit, peaking at 15.4 GB of utilized heap.**

More importantly, Eclat’s execution remained computationally structured and rhythmic. Rather than the chaotic GC thrashing that paralyzed the legacy system, the updated architecture executed the workload with a clean, sustained computational hill, peaking at a even higher **32%** CPU usage, **highlighting the core tradeoff of the Eclat Algorithm.**

Dataset Size	System / Algorithm	Algorithmic Time	Total Execution Time
64KB (SMCollection)	Legacy (Apriori)	1227ms	2148ms
	Current (Eclat)	19ms	143ms
40MB (20 News Groups)	Legacy (Apriori)	105ms	25,415ms
	Current (Eclat)	14ms	23,972ms
100MB (MIMIC-III)	Legacy (Apriori)	3477ms	16,792ms
	Current (Eclat)	808ms	12,179ms
500MB (Gutenberg)	Legacy (Apriori)	45,444ms	87,370
	Current (Eclat)	174ms	42,185ms
2.4GB (PDF Corpus)	Legacy (Apriori)	6746ms	11.13m
	Current (Eclat)	22ms	10.74m

12 Future Work

While the current iteration of the **Word Taxonomy Explorer** successfully resolves the computational bottlenecks of legacy association rule mining, a paramount achievement of this thesis lies in its **comprehensive visual and interactive overhaul**. By transitioning to a *dynamic*, component-based dashboard powered by physics-based rendering and real-time state injection, the system now provides a highly explanatory and *user-centric analytical environment*. Nevertheless, text analytics and data visualization remain **rapidly** evolving domains. The dual *architectural* and *visual* foundations established in this work provide a robust baseline for several advanced extensions. Future development of this system can be directed along **five primary vectors and a bonus one** to further enhance its analytical depth, scalability, and visual interpretability:

12.1 Semantic Expansion via Contextual Embeddings

Currently, the system relies on **exact lexical matching** facilitated by the *Eclat algorithm*. Consequently, synonymous or contextually related terms (e.g., "*automobile*" and "*car*") are treated as entirely distinct nodes, which can inadvertently fragment the semantic graph. A critical future enhancement involves integrating pre-trained contextual language models, *such as BERT or RoBERTa*. By mapping the extracted tokens into a dense, high-dimensional vector space, the system could perform a pre-clustering step using cosine similarity. This would allow the backend to merge highly synonymous terms into unified "*Concept Nodes*" prior to executing the frequent itemset mining, resulting in denser and conceptually richer taxonomies.

12.2 Distributed Mining Architectures for Big Data

Although the transition to the **Eclat** algorithm drastically improved in-memory computation speeds via *vertical data layouts* (Transaction ID intersections), the system currently operates as a **single-node application**. As the size of target corpora scales into the hundreds of gigabytes—a common scenario in enterprise-level data mining—single-machine **RAM limits will inevitably become a bottleneck**. Future iterations could adapt the backend to a **distributed computing framework**, such as *Apache Spark*. Because Eclat calculates support via **independent set intersections**, the algorithm is inherently parallelizable. Distributing these TID lists across a computing cluster using *Resilient Distributed Datasets (RDDs)* would enable the system to ingest, process, and map massive, enterprise-scale text repositories without memory degradation.

12.3 Advanced NLP and Temporal Graph Analysis

The current **Natural Language Processing pipeline** effectively isolates semantic terms using the *Stanford CoreNLP Part-of-Speech tagger*. Future development could incorporate advanced **Named Entity Recognition (NER) models** to explicitly categorize nodes, allowing the frontend to visually distinguish between *People*, *Organizations*, and *Locations* using distinct node shapes. Furthermore, if the ingested documents possess temporal metadata (such as publication dates), the system could be expanded to support **Dynamic Temporal Taxonomies**. This would allow researchers to track the evolution of concepts over time—for example, *observing how a specific term's Weighted PageRank centrality rises or falls across different years—utilizing an interactive timeline slider on the dashboard*.

12.4 Human-in-the-Loop (Active Learning) Integration

Given that the primary contribution of the frontend redesign is high interactivity, the next logical step is to **turn the user's interactions into a feedback loop for the backend**. Currently, the taxonomy is strictly machine-generated. By implementing a *Human-in-the-Loop (HITL)* approach, **users could manually curate the graph directly on the canvas** —such as *deleting anomalous edges, merging nodes, or manually boosting the confidence of a specific rule*. The frontend could then dispatch these corrections back to the REST API, allowing the system to update its internal static state and potentially apply Active Learning algorithms to prevent similar anomalies in future mining sessions.

12.5 Graph Interoperability and Exportation

Presently, the generated taxonomy is bound to the application's internal visualization engine via the `GraphDt` object. To increase the tool's utility for the broader research community, **a dedicated exportation module should be developed**. Permitting users to serialize the final `WeightedGraph` into standardized, open-source graph formats (*such as GraphML, GEXF, or JSON-LD*) would enable seamless interoperability. This would allow researchers to **export the mined relationships into dedicated graph databases** (*like Neo4j*) or advanced topological analysis software (*like Gephi*) for further algorithmic study outside the browser environment.

12.6 LLM-Driven Cluster Summarization via Retrieval-Augmented Generation (RAG)

The introduction of the contextual Evidence Modals successfully solved the "*Black Box*" problem by providing users with the raw text snippets justifying a mined rule. A highly ambitious future extension would be to elevate this explainability layer by **integrating Large Language Models (LLMs)**. By utilizing the system's existing snippet retrieval logic **as a foundation for Retrieval-Augmented Generation (RAG)**, the backend could feed these raw text segments into an LLM. On the frontend, instead of the user manually reading dozens of disjointed sentences to understand a semantic cluster, **the UI would present an AI-generated, abstractive summary explaining why those concepts are topologically grouped**. This would bridge the gap between strict statistical data mining and advanced natural language comprehension, establishing a truly intelligent visual analytics dashboard.

13 Bibliography

Bibliography

- [1] Rakesh Agrawal and Ramakrishnan Srikant. Fast algorithms for mining association rules in large databases. In *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB)*, pages 487–499. Morgan Kaufmann Publishers Inc., 1994.
- [2] Gökmen Altay, Neslihan Altay, and Cüneyt Güzelış. Improving the reliability of association rules: New metrics and analysis. *IEEE Access*, 10:34567–34580, 2022. Detailed analysis of Support, Confidence, and Lift for reliability.
- [3] Josh Barnes and Piet Hut. A hierarchical $O(N \log N)$ force-calculation algorithm. *Nature*, 324(6096):446–449, 1986.
- [4] H. Chen and K. Miller. The evolution of relevance: Why weighted pagerank outperforms standard heuristics in 2026. In *Network Science Review*, volume 9, pages 14–30, 2026. Supports the global ranking logic described in Section 3.4.2.
- [5] A. Derar et al. Dynamic adaptive thresholds for association rule mining: A stability-based approach. In *Journal of Data Mining and Knowledge Discovery*, volume 18, pages 112–128, 2025. Supports the Live Filtering and adaptive thresholds used in Section 4.4.2.
- [6] Y.G et al. Retrieval-augmented generation for large language models. In *arXiv preprint arXiv:2312.10997*, 2023.
- [7] Stephen Few. *Information Dashboard Design: The Effective Visual Communication of Data*. O'Reilly Media, Inc., 2006. Supports the transition to a Dashboard-Oriented Architecture in Section 4.2.
- [8] Linton C. Freeman. Centrality in social networks conceptual clarification. *Social Networks*, 1(3):215–239, 1978.
- [9] Thomas M. J. Fruchterman and Edward M. Reingold. Graph drawing by force-directed placement. *Software: Practice and Experience*, 21(11):1129–1164, 1991.
- [10] Tal Galili, Alan O’Callaghan, Jonathan Sidi, and Carson Sievert. heatmaply: an R package for creating interactive cluster heatmaps for modern data analysis. *Bioinformatics*, 34(9):1600–1602, 2018. Standard reference for interactive, cluster-based heatmaps in hierarchical data.
- [11] Design Trends Group. Narrative-driven dashboards and the role of micro-animations in data science. In *HCI Quarterly*, volume 31, pages 88–102, 2026. Validates the interactive "Spotlight Effect" and dashboard architecture in Section 4.2.
- [12] Zuguang Gu. Complex heatmap and chord diagram visualizations for big data integration. *Journal of Visualized Experiments (JoVE)*, 171:e62489, 2021. Modern perspective on using Chord Diagrams to visualize relational importance and flow.
- [13] Marti A. Hearst. Multi-paragraph segmentation of expository text. In *Proceedings of the 32nd annual meeting on Association for Computational Linguistics*, pages 9–16, 1994.

Foundational work on TextTiling and lexical cohesion, similar to your semantic chunking logic [cite: 251, 252].

- [14] Aidan Hogan et al. Knowledge graphs. In *ACM Computing Surveys (CSUR)*, volume 54, pages 1–37, 2021. The definitive modern reference for KG structure and ranking.
- [15] C. Innovations. Comparative efficiency of eclat and apriori in high-dimensional text mining. In *Journal of Computing Innovations and Emerging Technologies*, volume 12, pages 45–59, 2025. Validates the performance gains of the vertical data layout transition in Section 3.1.
- [16] Paul Jaccard. Étude comparative de la distribution florale dans une portion des alpes et des jura. *Bull. Soc. Vaudoise Sci. Nat.*, 37:547–579, 1901. Foundation of the Jaccard Similarity Coefficient used in Section 3.3.2.
- [17] Shaoxiong Ji, Shirui Pan, Erik Cambria, Pekka Marttinen, and S. Yu Philip. A survey on knowledge graphs: Representation, acquisition, and applications. *IEEE Transactions on Neural Networks and Learning Systems*, 33(2):494–514, 2021. Comprehensive modern overview of Knowledge Graph architectures.
- [18] Alistair E. W. Johnson, Tom J. Pollard, Lu Shen, Li-wei H. Lehman, Mengling Feng, Mohammad Ghassemi, Benjamin Moody, Peter Szolovits, Leo Anthony Celi, and Roger G. Mark. Mimic-iii, a freely accessible critical care database. *Scientific Data*, 3(1):160035, 2016.
- [19] Alistair E. W. Johnson, Tom J. Pollard, Lu Shen, Li-wei H. Lehman, Mengling Feng, Mohammad Ghassemi, Benjamin Moody, Peter Szolovits, Leo Anthony Celi, and Roger G. Mark. MIMIC-III Demo, a freely accessible critical care database. *Scientific Data*, 3:160035, 2016.
- [20] Efthimios Mitkousis and Yannis Tzitzikas. Interactive association map creation from documents using association rule mining. In *Communications in Computer and Information Science*, 2025.
- [21] Alessandro Moschitti. Text classification corpora. <https://disi.unitn.it/moschitti/corpora.htm>, 2003. Accessed: 2026-01-30.
- [22] Mark Newman. *Networks*. Oxford University Press, 2018. Covers noise resistance and local vs. global measures.
- [23] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation ranking: Bringing order to the web. *Stanford InfoLab Publication*, 1999. Foundational algorithm for the Importance Ranking module described in Section 3.4.2.
- [24] S. Pajankar and A. Kumar. Enhanced weighted pagerank for global node importance in knowledge networks. *Journal of Graph Algorithms and Applications*, 27(3):145–162, 2023. Specific to the "Global Weighted PageRank" used in your ranking module.
- [25] Tzitzikas Y. Papadakis, M. Answering keyword queries through cached sub- queries in best match retrieval models. In *Journal of Intelligent Information Systems*, pages 67–106, 2015.
- [26] Project Gutenberg. Free ebooks. <https://www.gutenberg.org>, n.d. Accessed: 2026-02-14.

-
- [27] Microsoft Research. Graphrag: A framework for combining knowledge graphs with generative ai. In *Microsoft Research Project Publications*, 2025. Foundational for the Future Work section regarding LLM-driven cluster summarization.
- [28] NVIDIA Research. Optimizing rag pipelines: The impact of advanced text chunking on llm recall. In *NVIDIA Technical Report Archive*, 2024. Provides empirical evidence for paragraph and semantic-level granularity.
- [29] Ben Shneiderman and Catherine Plaisant. *Designing the User Interface: Strategies for Effective Human-Computer Interaction*. Pearson Education India, 2010. Covers event-driven loops and explainable modal windows.
- [30] J. Smith and L. Wang. Dual semantic chunker (dsc): Improving retrieval quality through context-aware segmentation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 204–215, 2024. Relates to the Jaccard-based semantic chunking implemented in Section 3.3.2.
- [31] Akbar Telikani, Amir H. Gandomi, and Asadollah Shahbahrami. A survey of evolutionary algorithms for association rule mining. *Information Sciences*, 524:318–352, 2020. Modern perspective on ARM efficiency and metric optimization.
- [32] Wenpu Xing and Ali Ghorbani. Weighted PageRank algorithm. *Proceedings of the Second Annual Conference on Communication Networks and Services Research*, pages 305–314, 2004.
- [33] Mohammed J. Zaki, Srinivasan Parthasarathy, Mitsunori Ogihara, and Wei Li. New algorithms for fast discovery of association rules. In *Proceedings of the Third International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 283–286, 1997.